Development of a LabVIEW System for Rotating Detonation Engine (RDE) Research

by
Nathan Brady
nbrady@ncsu.edu

Supporting Faculty Mentor
Dr. James Braun
jamesbraun@ncsu.edu

Supporting Research Team Member
Frank Rice
fsrice@ncsu.edu

North Carolina State University
Mechanical Engineering (MS)
MAE586-601
Final Report

Raliegh, North Carolina
December 3, 2024

Table of Contents

List of Tables

List of Figures

Development of a LabVIEW System for Rotating Detonation Engine (RDE) Research Final Report

Author: Nathan Brady (nbrady@ncsu.edu)
Faculty Mentor: Dr. James Braun (jamesbraun@ncsu.edu)
Supporting Research Team Member: Frank Rice (fsrice@ncsu.edu)

1. Introduction

The Rotating Detonation Engine (RDE) has existed as a theoretical concept since the 1950s. However, it has only been in recent years that physical embodiments of these designs have been demonstrated successfully. The engine conceptually generates more power per unit of fuel burned by harnessing a continuous detonation of fuel rather than conventional deflagration. High fidelity simulations and physical prototypes have successfully demonstrated the desired power and efficiency gains from this method of fuel combustion [1]. The potential gains of RDE technology necessitate its development as a next generation rocket engine design.

Research into RDE design has found this new type of rocket engine to have unique problems. Previous research has uncovered issues such as flame front instability, need for optimum pressure ratios for engine thrust, need for novel fuel injector designs for optimum fuel and air mixing, and heat dissipation to name a few [2,3]. High fidelity simulations have been extensively explored to remedy some of these issues. Simulations of this nature prove extremely challenging given the incredibly dynamic nature of flame front propagation inside the engine. Thus, experimental observation is required to resolve the issues associated with RDE design.

NC State University has undertaken a pathfinder project to design, fabricate, and commission an RDE test stand for exploring the design challenges mentioned above. The scope of this project to be discussed in this report is the design, development, and testing of the LabVIEW control architecture that will automate the RDE test stand. Research topics will include Data Acquisition (DAQ) configuration for field device communication, user interface development for user operability, data collection for post-processing, and RDE device sequencing for automated engine control. The project concluded with a successful demonstration of analog input and digital output control capability. This checkout was a critical milestone towards an engine hot fire demonstration.

2. Methods

2.1 Materials

While the RDE test stand itself is made up of a significantly larger variety of components, this project is scoped to LabVIEW software, DAQ hardware, and instrumentation. The LabVIEW project was created using NI LabVIEW Professional Development System version 2024Q1 (National Instruments (NI), Austin, TX). DAQ configuration was prepared using NI MAX version 2023Q4. DAQ communication with LabVIEW was enabled using NI-DAQmx

Instrument Driver version 2024Q1. No additional external add-on packages were installed to complete programming for this project. All SubVI libraries were generated using standard LabVIEW functions as described in the methods and results section below. No third-party code libraries were utilized.

The DAQ unit was based on the modular instrumentation platform PXI Express (PXIe). This hardware was selected prior to the initiation of this LabVIEW implementation project. DAQ hardware selected includes items outlined in Table 2.1.1.

Table 2.1.1: DAQ Component Details

DAQ Component Model	**NI Component Description**	**Purpose**
PXIe-PCIe8398	PCI Host Interface Card	Enable communication between the NI chassis and LabVIEW software.
PXIe-6375	PXI Multifunction I/O Module	Primary DAQ module, containing AI, DIO and AO channels.
PXIe-6739	PXI Analog Output Module	AO expansion DAQ module.
PXIe-6535	PXI Digital I/O Module	DIO expansion DAQ module.
PXIe-4302	PXI Analog Input Module	Thermocouple conditioner DAQ module.
PXIe-1092	PXI Chassis	NI chassis for multi-DAQ module integration.

For conversion of the master instrumentation list into an initialization file, a MATLAB script file was generated using MATLAB version R2024a (MathWorks INC, Natick, MA). A similar MATLAB script file was generated for converting the automated test sequence file into a readable format for LabVIEW.

2.2 Methods

Prior development of the RDE test stand included design of an initial piping and instrumentation diagram (P&ID) to define an initial quantity of devices, layout of different flows, and nomenclature for each respective device. The P&ID developed is shown in Figure 2.2.1. The standard naming convention shown in this P&ID is defined in Table 2.2.1.

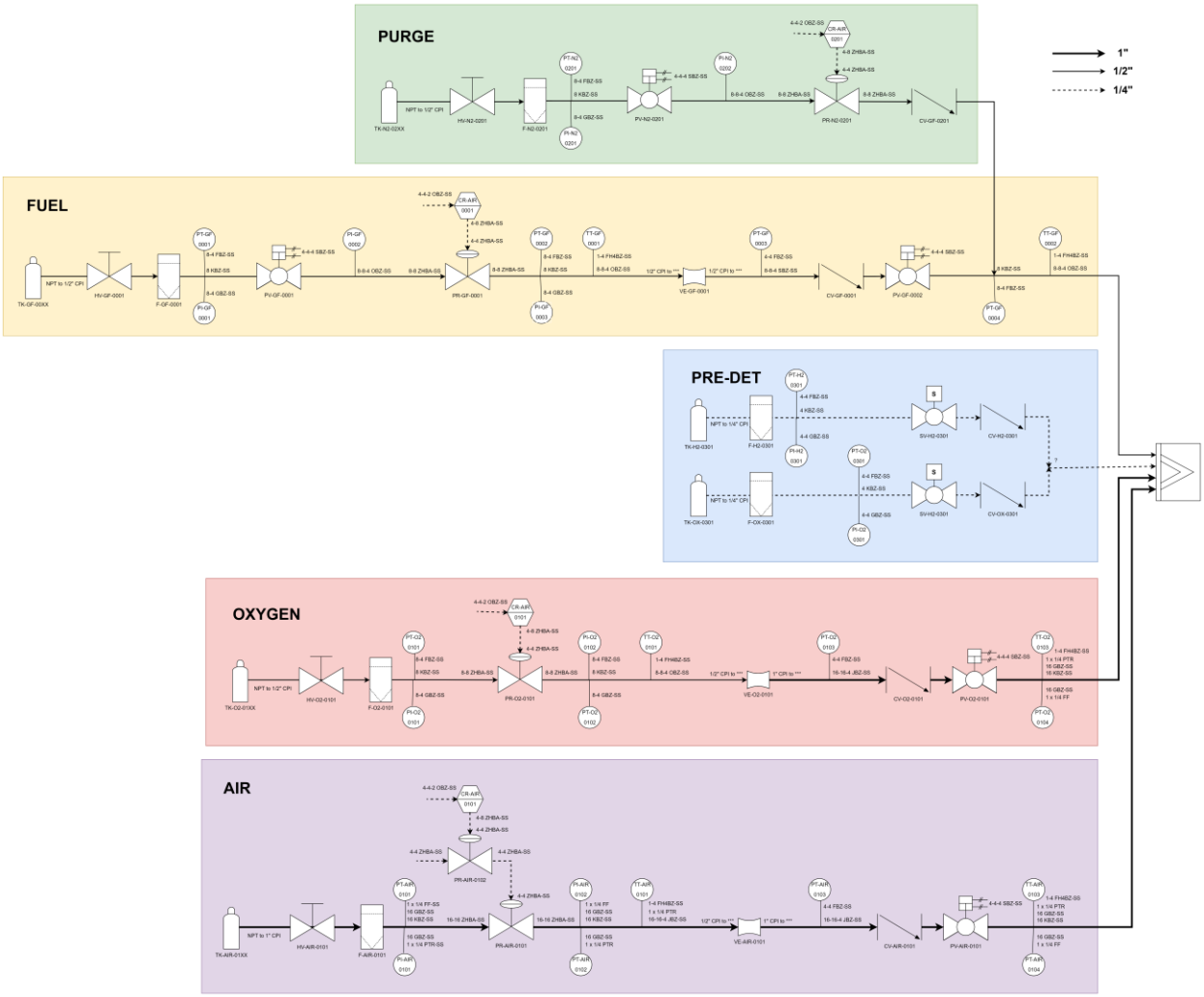


Figure 2.2.1: Early P&ID Design

Table 2.2.1: P&ID Labeling Scheme
Standard Format: AA-BB-CDEE

AA (hardware type)	**BB (media type)**	**C (system identifier)**	**D (common line)**	**EE (unique ID)**
TK: tank HR: hand regulator CR: computer regulator HV: hand valve PV: pneumatic valve SV: solenoid valve VE: venturi PT: pressure transducer PI: pressure indicator TT: temperature transducer TI: temperature indicator F: filter LC: load cell	N2: Nitrogen OX: Oxygen GF: gaseous fuel AIR: air RDE: engine	0: facilities 1: small test stand 2: liquid fuel injection rig	Common line identifier	Unique identifier, ascending from gas bottle to engine

To bring the information associated with each field device into LabVIEW, a Master Instrumentation List (MIL) was generated in Microsoft Excel. This instrumentation list was designed to contain all meta data associated with each field device. For this report, a field device was defined to be any sensor, valve, or communicating component that provides input to or receives output from the DAQ. Each field device was assigned a unique channel. Each channel contains the meta data associated with each device. The meta data used for each channel was dependent on the type of channel it was designed to be – analog input (AI), analog output (AO), digital input (DI), digital output (DO), or thermocouple (TC). However, for consistency, each field device was assigned the same meta data Sections and Keys as defined by Table 2.2.2. The MIL also included additional information about each field device such as descriptions and system locations to provide clear device documentation.

Table 2.2.2: Master Instrumentation List Section and Keys*

Information	**Units**	**Example**	**Source/Purpose**
Device ID	-	PT-N2-0001	P&ID unique device identifier, Section identifier
Signal type	-	AI	Type of input or output
Terminal configuration	-	Differential	Define wiring of the device into the channel terminals
Device identifier	-	PXIe-6375	PXIe card the device is wired to
Channel	-	ai0	Channel the device is wired to
Signal unit	-	Volts	Raw unit of measure
Minimum	V	0	Minimum expected input or output signal
Maximum	V	10	Maximum expected input or output signal
Calibration zero offset	V	0.001	Sensor calibration zero offset
Calibration slope	V/V	1.001	Sensor calibration slope
Engineering scale	psi/V or C/V	200	Scale for conversion to engineering units

*An additional 3 spare section keys were included to ease future expansion

To retrieve the MIL information into LabVIEW, a MATLAB script file was prepared to convert the MIL data into an initialization text file for LabVIEW to easily read. This MIL to initialization file arrangement allows for easy reconfiguration of field device information into LabVIEW without manually recoding the LabVIEW software for each run.

Similar to the MIL, a test sequence file was generated in Microsoft Excel to codify a sequence of device events to operate automatically when triggered by the user. Each line of the test sequence identifies an event time in milliseconds from time zero. It specifies what output device (e.g. regulator, valve, camera) to control based on its device ID and provides the new value to assign to the device. At the event time, the device is assigned the new value. Each line occurs one at a time in sequence; thus, only one device can be changed at a time. However, to perform multiple actions simultaneously, identical event times can be provided for each line. This will instruct the system to fire each device as quickly as possible in the order provided. To visualize the test sequence, a graphical representation of the critical valve positions was created to provide visual feedback of the automated sequence. This allows the operator to see critical steps in the process to avoid unsafe operation caused by incorrect sequence programming. The test sequence is loaded into LabVIEW in a similar manner to the MIL using a MATLAB script file to convert the Excel file format into an initialization text format for LabVIEW to read.

With MIL and test sequence information prepared in a format easily readable with LabVIEW, a coding structure was developed to manage inflow and outflow of data between the RDE and user. At a very high level, the LabVIEW system was designed to 1) input the MIL and test sequence data, 2) establish communication with the DAQ unit, 3) maintain live measurements with field devices, 4) define an automated RDE actuation sequence, and 5) provide a shutdown sequence to disconnect the DAQ when complete. The general data flow is visualized in Figure 2.2.2.

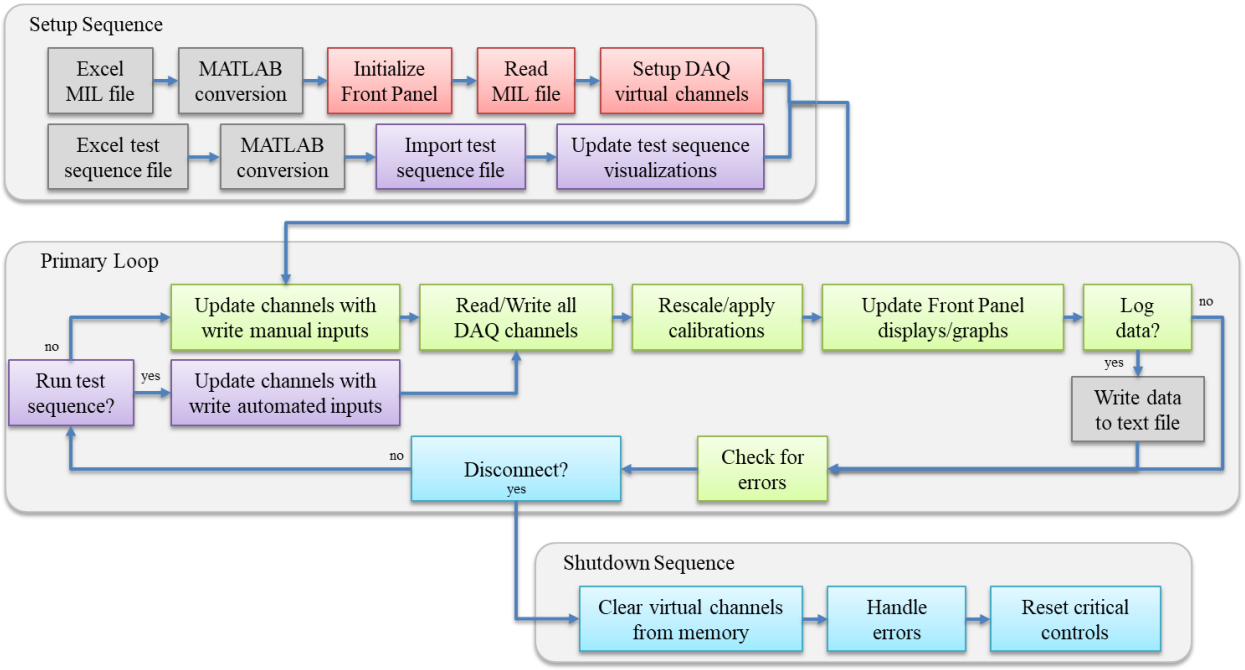


Figure 2.2.2: High Level Process Diagram

To simplify repeated tasks within the code, a library of SubVIs was developed to handle various aspects of data processing. The general purpose of each SubVI is summarized in Table 2.2.3. The top-level MainVI incorporated these SubVIs together into a single cohesive user interface. The front panel of this user interface was designed with the intent for future flexibility. Sensor indicators were constructed such that device IDs can be changed directly from the front panel based on label assignments. Device indicators and controls can be rearranged in a variety of configurations on the front panel without significant modification to the controlling block diagram. Data is assigned to the correct front panel display location based on its P&ID defined device ID.

Table 2.2.3: Summary of SubVIs

SubVI name	**Purpose**
subVI_read_configFile.vi	Reads the MIL configuration file as prepared by the MATLAB conversion script. Used once at the start of each MainVI session.
subVI_initialize_PhysicalChannels.vi	Creates physical channel assignments and initializes communication with the DAQ. Used once for each MainVI session.
subVI_search_ReadChannels.vi	Parses the stream of virtual channels and displays measured input data. Used continuously during MainVI operation.
subVI_search_WriteChannels.vi	Parses the stream of virtual channels and outputs user setpoints into the data stream for writing to output channels. Used continuously during MainVI operation.

SubVI name	Purpose
subVI_apply_CalibrationAndScaling.vi	Parses the virtual channels and applies user specified calibration and scaling information to each input channel's live data. Alternatively scales output data setpoints into raw DAQ units. Used continuously during MainVI operation.
subVI_readWrite_AllChannels.vi	Reads all DAQ inputs and writes to all DAQ outputs according to each virtual channel's latest data stream. Used continuously during operation to provide constant communication between the MainVI front panel and DAQ.
subVI_write_DataToFile.vi	Records channel data and time stamp information to an output data file for post processing. Used only as indicated by the user or automated test sequence.
subVI_read_testSequence_File.vi	Reads the test sequence file as prepared by the MATLAB conversion script. Used each time as called by the operator.
subVI_graphTestSequence.vi	Reads the test sequence tabulated data as displayed in LabVIEW and updates a test sequence visualization. Used each time as called by the operator.
subVI_GraphPressureTrends.vi	Provide pop-up window to graph various outputs. Graphs data based on user specified device IDs. Used continuously during operation.
subVI_GraphTemperatureTrends.vi	Provide pop-up window to graph various outputs. Graphs data based on user specified device IDs. Used continuously during operation.
subVI_clear_Tasks.vi	Destroys virtual task assignments in computer memory at the end of each MainVI session execution to prevent memory overwrite errors.

SubVIs and the MainVI are contained within the context of a LabVIEW Project. This project file acts as a file directory to define active SubVIs for the MainVI to call as needed. It provides the framework in which the program operates. The LabVIEW project containing the various SubVIs is shown in Figure 2.2.3.

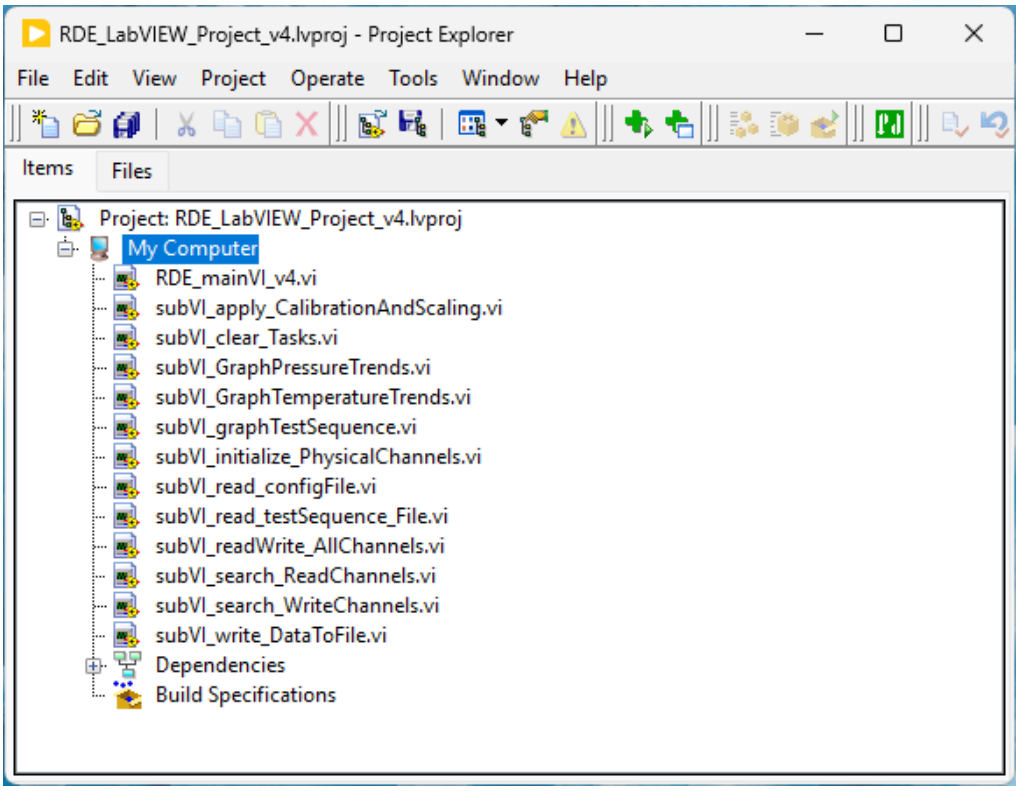


Figure 2.2.3: LabVIEW Project Directory

3. Results

3.1 MainVI Architecture

The MIL was pre-populated with an initial configuration of devices as displayed in the P&ID. It was expanded to include other devices such as pressure transducers on the RDE itself, camera trigger, load cell, and valve position feedback channels. An initial feasible set of physical channel assignments for each device was identified in the MIL. It is noted that this initial selection of channels will likely be changed as physical wiring of the larger system progresses. The MATLAB script file to convert the MIL to an initialization file was successfully created and implemented. A screenshot of the initial MIL is provided in Appendix A Figure A1.

Conversion of raw sensor inputs to calibrated engineering units is supported through the MIL. Calibrations for each sensor are supplied to the MIL and read into LabVIEW via the configuration file. Currently, a linear calibration scheme of zero offsets and linear calibration slope constants are uniquely assigned to each channel. However, higher order calibration curves could be implemented if required. A scaling constant to convert from raw measurement units to engineering units is also supplied for each sensor channel. The calibration constants and engineering scaling factor are applied to the raw measurements in the applicable “SubVI_apply_CalibrationAndScaling” SubVI.

An initial test sequence file was prepared in Microsoft Excel to serve as a template for future test sequences. A corresponding MATLAB script file was also prepared to convert the test sequence file to an initialization file for reading into LabVIEW. The capability of these files was tested and found suitable for use. A screenshot of the test sequence template is provided in Appendix A Figure A2.

NI MAX was used to configure the PXIe chassis and subsequent DAQ modules. The actual devices were configured to be identified by their generic card name (e.g. PXIe-6375). To enable offline testing of LabVIEW software updates and safe testing of automated test sequences, a mirrored set of simulated DAQ devices were configured by the same naming convention but defined with “sim” in the name (e.g. PXIe-6375sim). For future expansion, it is noted that naming modification is managed in NI MAX. Figure 3.1.1 provides a screenshot of the device naming convention as shown in NI MAX.

It is noted that tasks were not assigned to the virtual channels through NI MAX. While this capability exists, it was opted that DAQmx task assignment would be completed programmatically within the MainVI and SubVIs. This route was chosen to enable the MIL to control channel assignment.

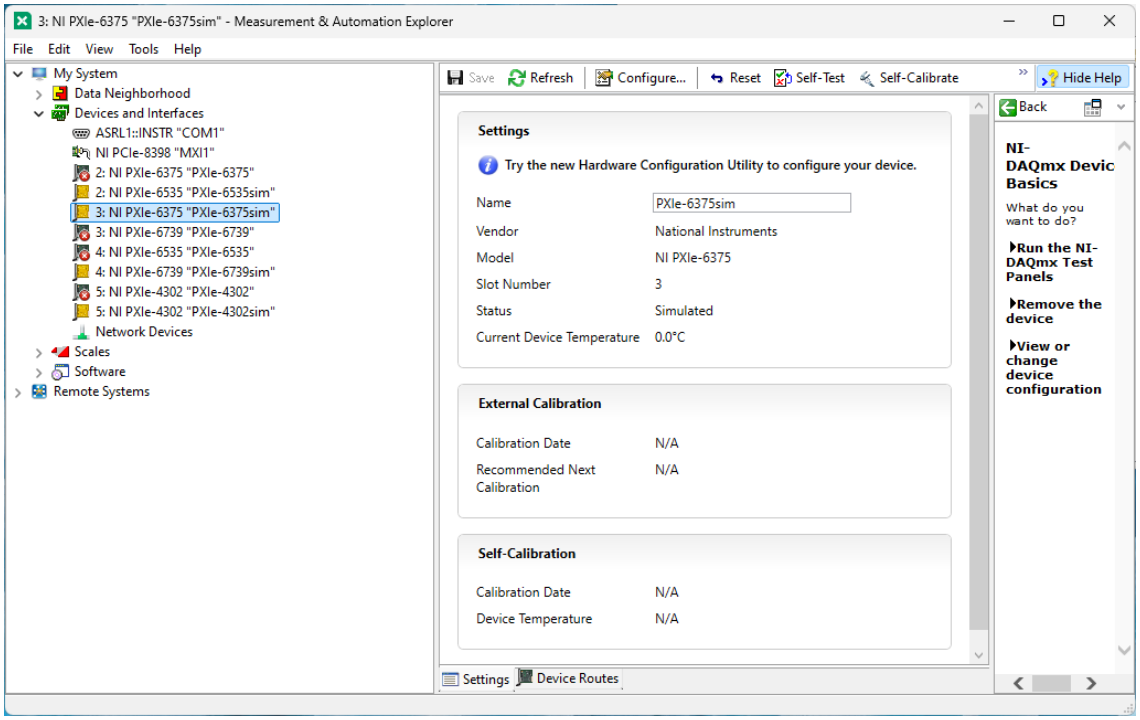


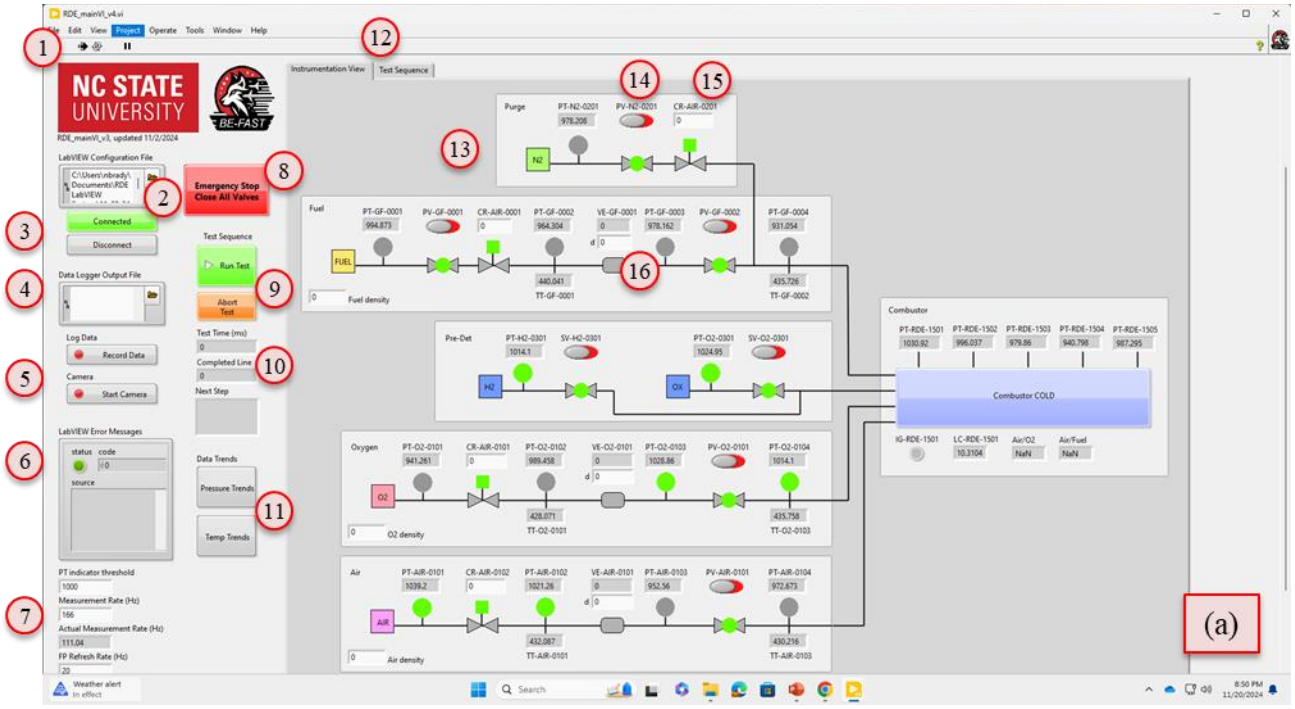
Figure 3.1.1: NI MAX DAQ Module Naming Configuration (simulated device highlighted)

All SubVIs outlined in Table 2.2.3 were created successfully in LabVIEW. The primary thread tying the entire program together is the array of virtual channels created out of the “read_configFile” SubVI. This array is a 1D array of clusters. The array size is equal to the number of channels defined by the MIL, and thus each array element corresponds to each field device. Each array index contains one cluster of elements that comprise the meta data associated with that field device. Physical channel assignments, tasks, measurement data, calibration

information, and other such meta data are carried in each cluster. Each virtual channel can be called and manipulated according to its unique device ID as defined by its P&ID naming convention. The device IDs are assigned to each virtual channel through the MIL.

The Main VI and critical SubVIs were equipped with error handling capability to manage potential error generation. This is considered standard practice for LabVIEW programs. Error signals were used to identify if and where errors appeared, and error handling functions were implemented to provide the user with feedback for troubleshooting.

The final design of the MainVI was arranged to mimic the primary P&ID structure. This structure was chosen to provide quick, clear feedback to the user of each device's state. The MainVI Front Panel is shown in Figure 3.1.2a and b. A high-level overview of the block diagram for the MainVI is shown in Figure 3.1.2c. Key items are flagged on these figures with descriptions provided in Table 3.1.1.



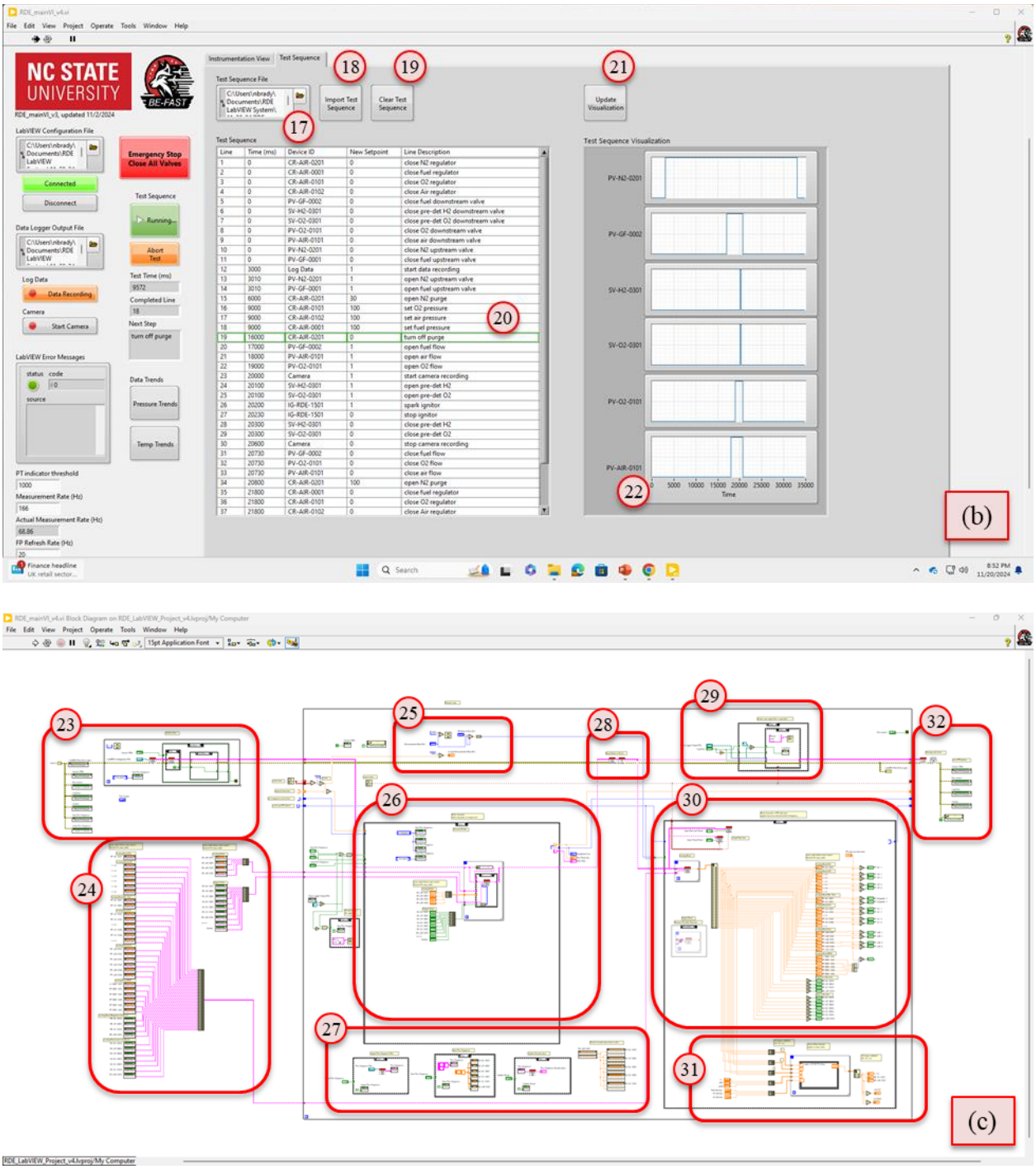


Figure 3.1.2: RDE MainVI Front Panel (a) Instrumentation Tab, (b) Test Sequence Tab, and (c) RDE MainVI Block Diagram

Table 3.1.1: MainVI Key Items

Item	Description	Function
1	LabVIEW run button	Transitions LabVIEW from editor mode to Run mode.
2	MIL configuration file path	Required input to configure device names and associate with DAQ physical channels. Also imports device calibration and scaling information.
3	Connect to / Disconnect from PXIe control buttons	Control the connection to the DAQ. Only allows connection after configuration file is provided. Only allows disconnection if not running an automated sequence.
4	Data logger output file path	Specifies file path location for logged data. Data will be saved as a tab delimited text file.
5	Log Data and Camera toggle buttons	Toggle ON/OFF status of the data logger and camera. Can be actuated manually or automatically as part of a test sequence.
6	Error status monitor	Provides live status of errors present in the system and associated error code and message.
7	Loop iteration speed controls	Control and monitor primary loop iteration frequency and front panel refresh frequency.
8	Emergency stop button	Control button to simultaneously close all valves and regulators programmatically in the event of any undesired system behavior.
9	Test sequence control buttons	Control buttons to activate loaded test sequence. Abort button performs similar shutdown sequence as emergency stop button.
10	Test sequence status monitors	Indicators of test sequence timer and active test sequence task to track automated activity.
11	Data trend chart pop-up window control buttons	Toggle buttons to pop-up secondary windows for control charts. Charts will open each time this button is pressed.
12	Tab display control buttons	Tab control to switch view from live instrumentation view to test sequence page. Pages are always both active, but only one is visible at a time.
13	P&ID highlighted control group	Highlighted areas group sections of controls. Highlighting is purely decorative to improve readability. Controls and indicators can be placed anywhere on the screen.
14	Digital output, valve control	Example of valve control switch. Can be manually toggled ON/OFF while in manual mode. Will automatically change position when called during a test sequence.
15	Analog output, regulator control	Example of a regulator control input. Desired pressure setting can manually be assigned in the box. Will automatically change value when called during a test sequence.

Item	Description	Function
16	Venturi inputs and calculated mass flow rate display	Venturi throat diameter input and calculated mass flow rate. Mass flow rate calculated based on upstream inputs.
17	Test sequence input file path	Specifies file path location for the test sequence initialization file.
18	Import test sequence toggle button	Imports the test sequence file after the file path has been specified.
19	Clear test sequence toggle button	Clears all test sequence data from test sequence table display.
20	Test sequence table display	Displays imported test sequence data. Can be manually modified after loading. Test sequence control will run what appears on this table. The import test sequence can be used to overwrite/reset undesired changes. During run execution, the next line to be executed is highlighted in green.
21	Test sequence update visualization toggle button	Reads the test sequence table and provides a time history plot of the critical valve positions. If a modification is required, the test sequence table can be updated and visualization updated.
22	Test sequence visualization time scale controls	Test sequence visualization time scale can be modified on the x-axis to zoom in on a desired time frame. Resetting to Autoscale will refit the plot to the full test sequence time scale.
23	Block diagram program initialization section	Initializes critical control parameters, checks for an active MIL file, initiates DAQ connection.
24	Block diagram device naming section	Pulls names of all front panel device labels. Device labels on front panel must match corresponding device IDs assigned in MIL.
25	Block diagram loop timing controls section	Sets loop control and front panel update frequencies based on user specified inputs.
26	Block diagram output signal controls section	Control structure to write analog and digital outputs. Switches between manual and automatic control as indicated by the user. Defaults to manual mode.
27	Block diagram test sequence file controls section	Code controls for test sequence file loading and visualization.
28	Block diagram DAQ communication SubVIs	SubVI for communicating outputs to and reading inputs from the DAQ. Rescaling SubVI to apply calibrations and scaling factors to each channel.
29	Block diagram data logging section	Control structure to log data. Interlocks to prevent file creation errors.
30	Block diagram input signal indicators section	Control structure to read analog and digital inputs to the front panel.

Item	Description	Function
31	Block diagram venturi calculations section	Control structure to calculate mass flow rate from associated inputs.
32	Block diagram shutdown controls section	Control structure to disconnect DAQ and clear virtual channels from memory. Resets critical front panel controls.

The system was debugged and interlocked to provide the user with seamless equipment control. The user is able to manually control all outputs to the system and monitor all inputs in real time. Critical control buttons are interlocked to prevent undesirable system behavior (i.e. the user is prevented from disconnecting the DAQ in the middle of an automated sequence). All equipment can be verified from this manual mode to prepare the system for an automated test sequence.

The automated test sequence was also debugged to provide smooth operation. The user is able to load a test file of any length and specify any controllable device ID to a new value. The test sequence is designed to initiate at whatever state the system was left in manual mode. This provides the user with full flexibility to automate the entire test protocol from a completely dormant system state or actuate only a small portion of the high-speed valve actuation from a manually prepared system state.

By loading a simulated MIL file and utilizing the simulated DAQ devices configured in NI MAX, the user can fully simulate operation of the system including test sequence operation. This enables full checkout of an intended test sequence to identify accidental test sequence operations before they lead to an unsafe event.

The trend chart pop-up SubVIs were created to enable continuous visualization of any device of interest. They were designed for full user flexibility such that any device can be monitored on any available chart in any order. Each chart can accommodate up to 12 devices at once and will by default display approximately one minute of buffered data. Figure 3.1.3 provides a screenshot of one of these pop-up windows with a few representative devices enabled.

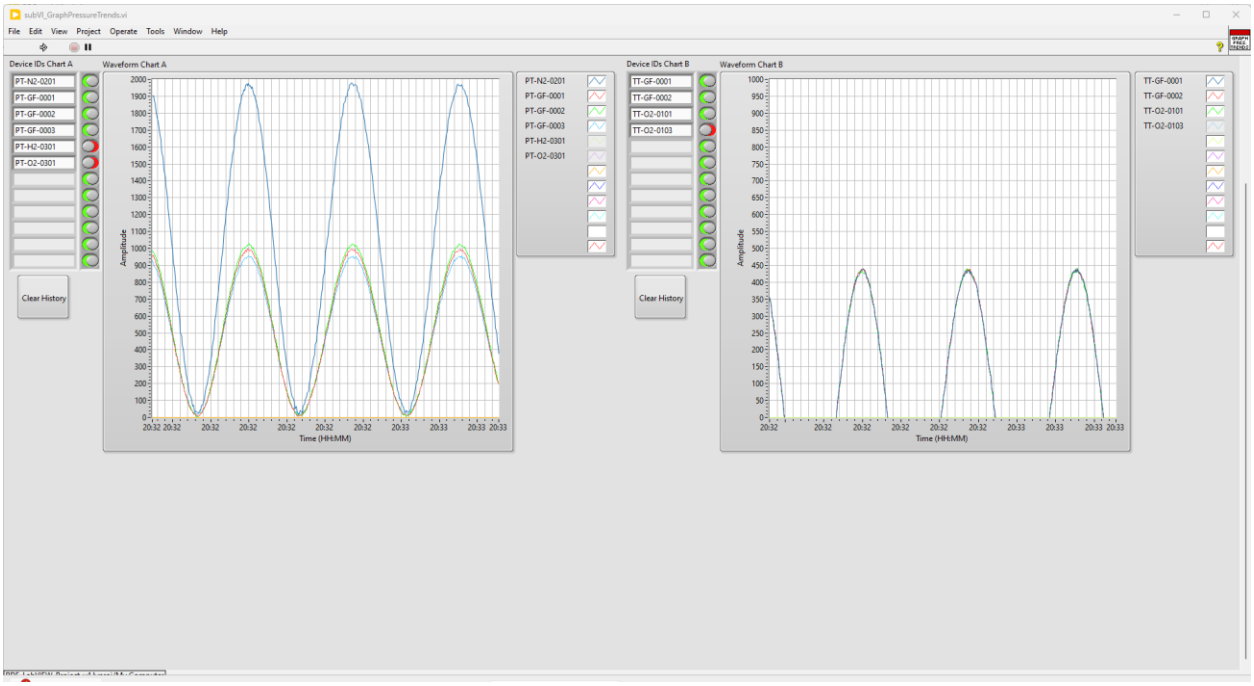


Figure 3.1.3: Trend Chart Pop-up Window (arbitrary representative devices displayed)

3.2 SubVI Architecture

The library of SubVIs that were created are described here in terms of their inputs and outputs as they relate to the larger MainVI. The purpose of each SubVI is described in detail. A screenshot of the block diagram for each is also provided for clarity.

3.2.1 subVI_read_configFile.vi

This sub routine is designed to read the initialization file into LabVIEW and parse out each element of information and relate it to each device ID. Each device ID is assigned a signal type, terminal configuration, device identifier (PXIe module), channel, unit of measure, signal span, signal calibration constants, and engineering scale conversion constant. These attributes comprise a single cluster associated with the device ID. This is repeated for each line of the MIL such that an array of clusters is generated to describe all devices. This SubVI is designed to run only once at the beginning of each LabVIEW run session. To make changes to the MIL, the user must disconnect the DAQ and reload a new MIL file before reconnecting to the DAQ. Table 3.2.1 describes this SubVIs inputs and outputs. Figure 3.2.1 provides a screenshot of the block diagram for this SubVI.

Table 3.2.1: SubVI_read_configFile.vi Inputs and Outputs

Inputs	Outputs
Configuration file path – file path to the initialization file built by MATLAB that was generated from the configuration file excel template.	Configuration cluster – an array of clusters where each cluster comprises one device line from the MIL. The array comprises the entire MIL device dataset.
Error in – standard error cluster comprised of error indicator, error code, and error message.	Error out – standard error cluster comprised of error indicator, error code, and error message.

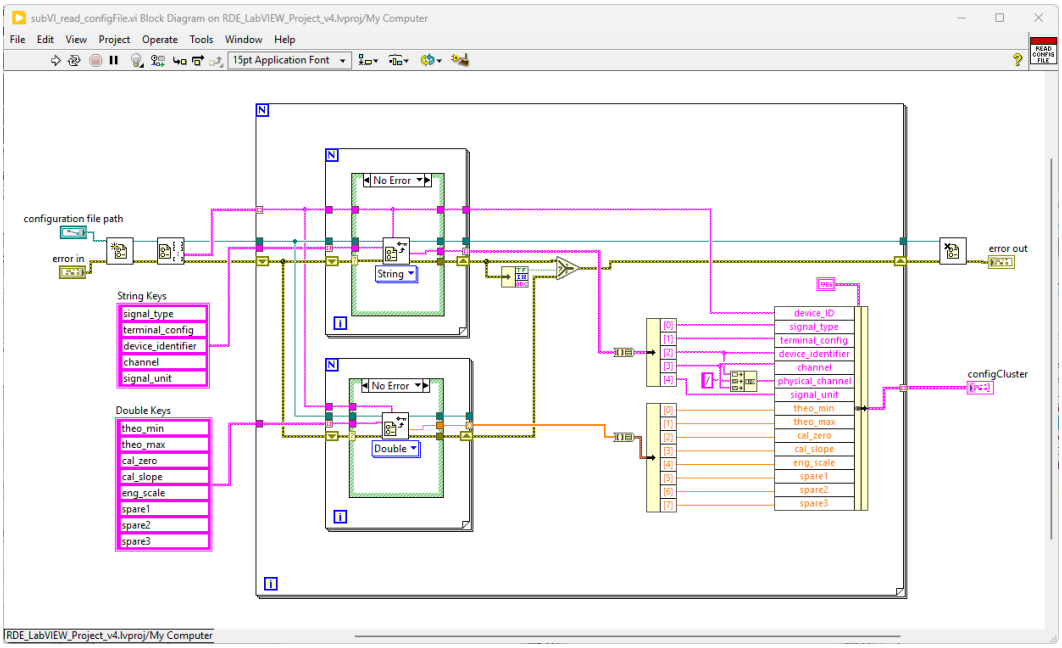


Figure 3.2.1: Read Config File SubVI Block Diagram

3.2.2 subVI_initialize_PhysicalChannels.vi

This subroutine loops through each device ID to create a new task. This task carries channel configuration information for each device and is used by the DAQmx driver to communicate to the DAQ to either input or output information. Tasks can be constructed for each individual channel or be created as a group of channels. For this system design, tasks were assigned on an individual basis to mirror the MIL structure and allow every device to be configured independently. Table 3.2.2 describes this SubVI's inputs and outputs, and Figure 3.2.2 shows its block diagram.

As will be discussed further in the test results section, this SubVI may require further development and a transition to handling of channels in a grouped structure. More specifically, it was found that it is preferable to group devices and channels on a per DAQ module basis. This allows communication with each DAQ module to be triggered to start and stop outside the primary loop to enhance loop execution speed.

Table 3.2.2: SubVI_initialize_PhysicalChannels.vi Inputs and Outputs

Inputs	Outputs
Configuration cluster – array of clusters from initialization file SubVI.	Virtual Channels out – very similar array of clusters to the configuration cluster. This array of clusters defines a task for each device ID according to its configuration.
Error in – standard error cluster comprised of error indicator, error code, and error message.	Error out – standard error cluster comprised of error indicator, error code, and error message.

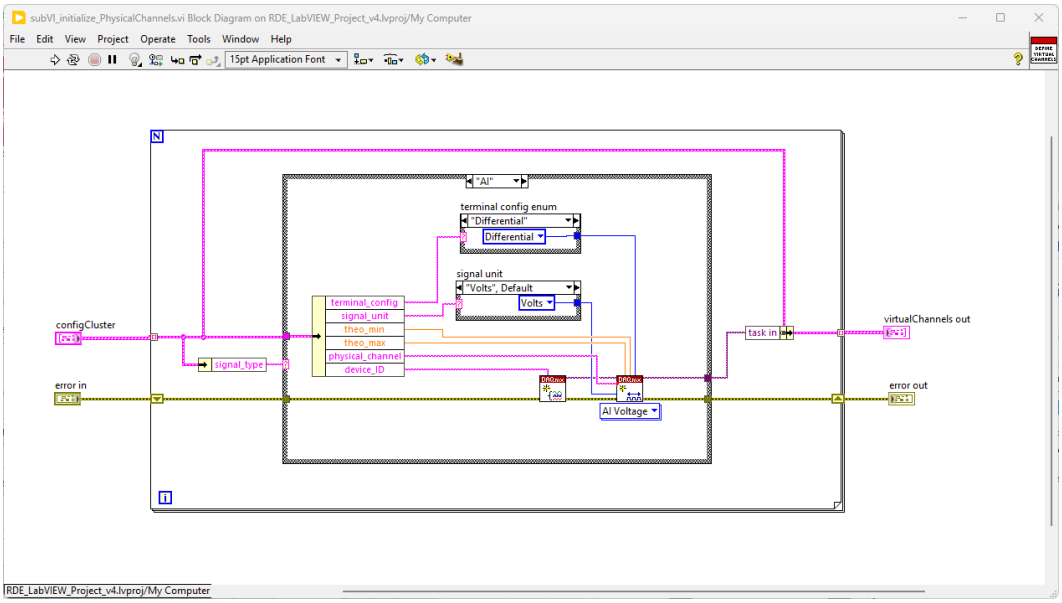


Figure 3.2.2: Initialize Physical Channels SubVI Block Diagram

3.2.3 subVI_search_ReadChannels.vi

This simple subroutine is designed to be reused throughout the MainVI and even within other SubVIs. It inputs the array of virtual channels with all their associated data. It then searches these virtual channels for a specific device ID. Upon finding it, the SubVI returns the current scaled data associated with that device ID. This is the primary subroutine to pull data from the virtual channels according to a specified device ID. Table 3.2.3 describes this SubVI's inputs and outputs, and Figure 3.2.3 shows its Block Diagram.

To search for multiple device IDs simultaneously, this SubVI can be placed in a for loop where an array of device IDs is auto indexed as an input. The for loop will then auto index a matching set of data in the same order as the device IDs supplied. However, when using this SubVI in this manner, it is critical to disable auto indexing of the virtual channel array of clusters. The virtual channels must all be available for searching.

Table 3.2.3: SubVI_search_ReadChannels.vi Inputs and Outputs

Inputs	Outputs
Virtual Channels in – array of clusters for all devices.	Scaled data – returns scaled data for the device ID specified.
Device ID to find – single device ID to find scaled data for.	

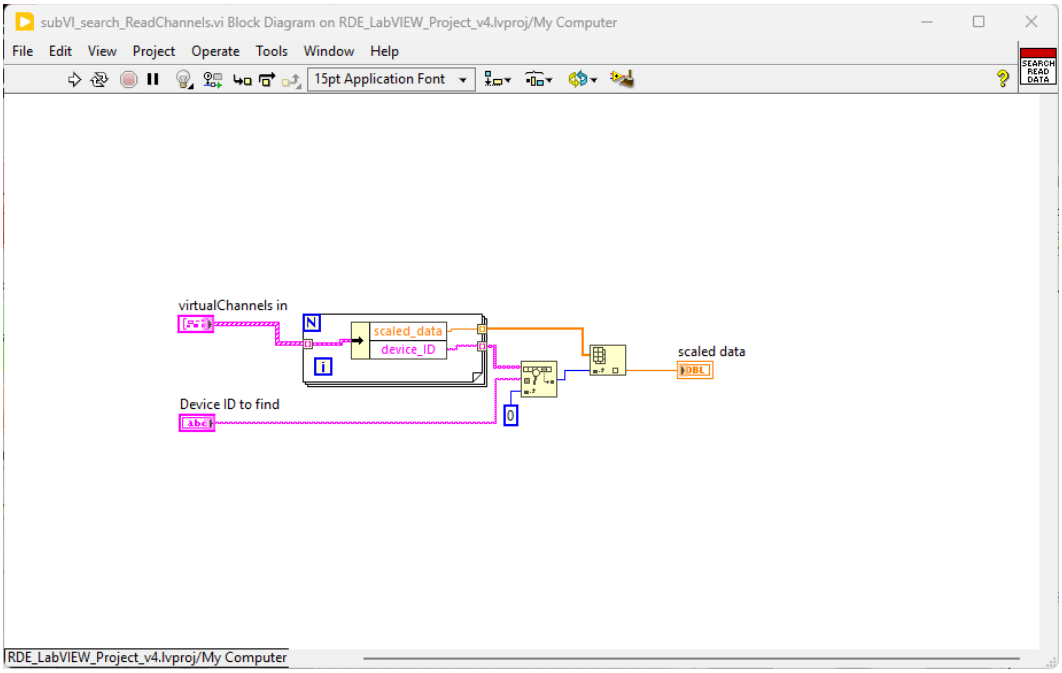


Figure 3.2.3: Search Read Channels SubVI Block Diagram

3.2.4 subVI_search_WriteChannels.vi

This subroutine performs the opposite action of the Search Read Channel SubVI. Instead of searching the entire array of virtual channels for a single device ID to read, this SubVI compares a single element of the virtual channel array against an array of device IDs to be written. If the virtual channel device ID is found among the array of devices to be updated, its corresponding scaled data is written to the virtual channel.

This SubVI is most typically used inside a for loop to search each element of the virtual channel array for a desired update. Contrary to the read SubVI, the virtual channel array should be auto indexed in this for loop while the searching device IDs and corresponding data arrays should not be auto indexed as inputs to the for loop. This behavior is exactly opposite of the Search Read Channel SubVI behavior. Table 3.2.4 describes this SubVI's inputs and outputs, and Figure 3.2.4 shows its Block Diagram.

Table 3.2.4: SubVI_search_WriteChannels.vi Inputs and Outputs

Inputs	Outputs
Single virtual channel in – a single channel cluster from the virtual channel array of clusters.	Single virtual channel out – a single channel cluster. If the device ID of this channel matches any of the device IDs provided, its scaled data will be rewritten. Otherwise, it is passed through without update.
Device ID data to write – an array of new data to be written to the associated array of device IDs provided. Data to be provided in scaled engineering units.	
Device IDs to find – array of device IDs to have new data assigned to them	

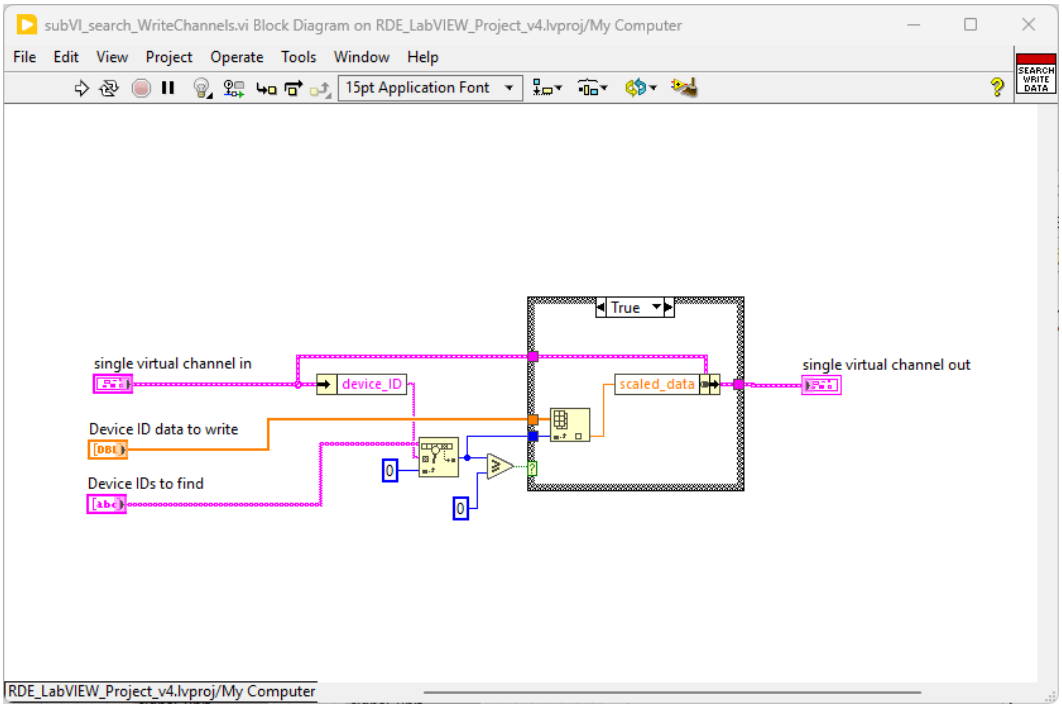


Figure 3.2.4: Search Write Channels SubVI Block Diagram

3.2.5 subVI_apply_CalibrationAndScaling.vi

This subroutine is designed to run once with each cycle of the MainVI loop. It converts input and output signals from DAQ raw data units (e.g. volts) to engineering scaled data units (e.g. psi, °C, or lbs.). During the conversion between raw data units to scaled data units, a set of linear calibration constants are applied to the raw signal including a slope correction and constant zero offset. This scaling and recalibration action acts on both inputs and outputs, although the calculations are the inverse of each other depending on the signal type. Scale and calibration information for each device ID is defined in the MIL on an individual device basis. While calibrations cannot be updated live, they can easily be updated in a new MIL file and reloaded

into LabVIEW. Table 3.2.5 describes this SubVI's inputs and outputs, and Figure 3.2.5 shows its Block Diagram.

Table 3.2.5: SubVI_apply_CalibrationAndScaling.vi Inputs and Outputs

Inputs	**Outputs**
Virtual channels in – array of clusters for all devices.	Virtual channels out – array of clusters for all devices.
Error in – standard error cluster comprised of error indicator, error code, and error message.	Error out – standard error cluster comprised of error indicator, error code, and error message.

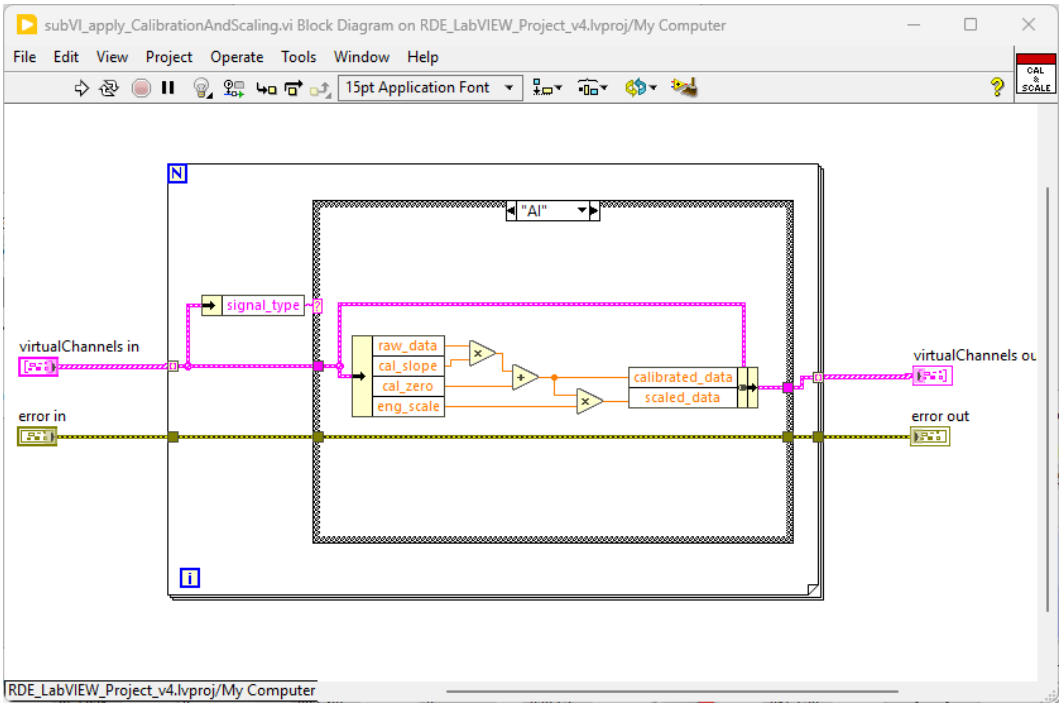


Figure 3.2.5: Apply Calibration and Scaling SubVI Block Diagram

3.2.6 subVI_readWrite_AllChannels.vi

This subroutine is designed to run once with each cycle of the MainVI loop. It performs the actual communication with the DAQ via the DAQmx driver. It writes analog and digital output data to the DAQ and retrieves analog and digital inputs from the DAQ. This is done on an individual channel basis based on each virtual channel's task. This SubVI's operation is closely tied to the architecture defined by the Initialize Physical Channel SubVI.

As mentioned in the Initialize Physical Channels SubVI, the choice of performing this activity on an individual channel basis was found to not be ideal. By performing each task for each channel individually, the DAQ is forced to start and stop acquisition for each channel within the primary loop which ties up resources and ultimately reduces communication speed. It is speculated that a grouped channel task based on each DAQ module could greatly mitigate communication delays. Table 3.2.6 describes this SubVI's inputs and outputs, and Figure 3.2.6 shows its Block Diagram.

Table 3.2.6: SubVI_readWrite_AllChannels.vi Inputs and Outputs

Inputs	Outputs
Virtual channels in – array of clusters for all devices.	Virtual channels out – array of clusters for all devices.
Error in – standard error cluster comprised of error indicator, error code, and error message.	Error out – standard error cluster comprised of error indicator, error code, and error message.

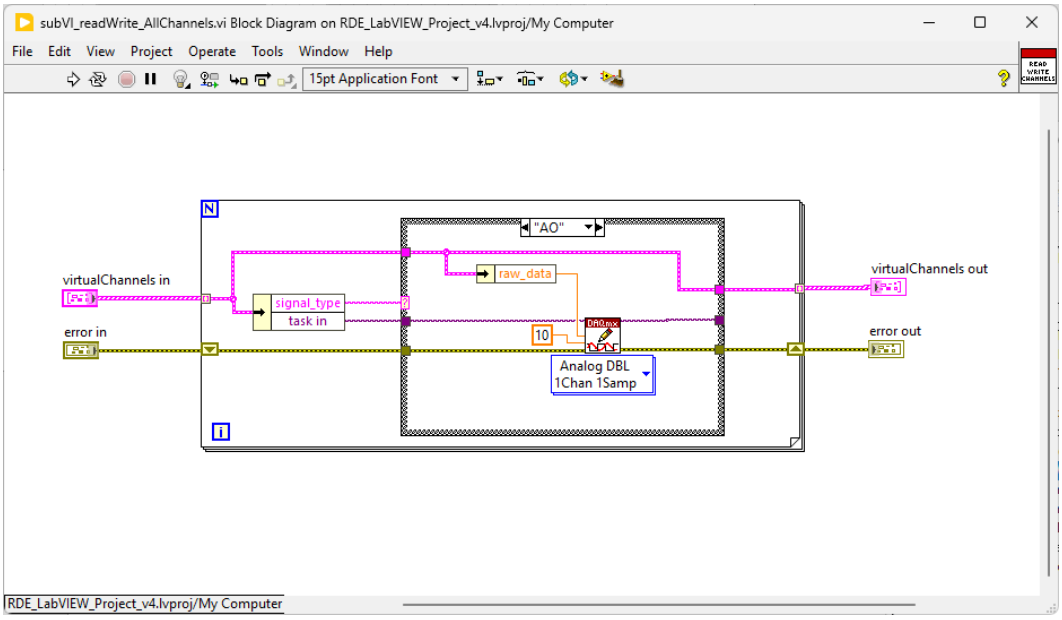


Figure 3.2.6: Read and Write All Channels SubVI Block Diagram

3.2.7 subVI_write_DataToFile.vi

This subroutine records all channels loaded into the MIL when specified by the user. Datalogging can be triggered manually by the user from the front panel or triggered automatically during a test sequence. Interlocks prevent the user from attempting to run this SubVI without first specifying a file path. The file generated is a tab delimited text file that contains a column for the date, time, and value of each device ID. A header row is provided on the first row for clarity. Time stamp data is recorded to the nearest millisecond. Table 3.2.7 describes this SubVI's inputs and outputs, and Figure 3.2.7 shows its Block Diagram.

Table 3.2.7: SubVI_write_DataToFile.vi Inputs and Outputs

Inputs	Outputs
Start recording – Boolean indicator to start or stop recording.	Virtual channels out – array of clusters for all devices.
New recording – Boolean indicator to indicate a new recording. If true, a header line containing device IDs will be applied to the file.	Error out – standard error cluster comprised of error indicator, error code, and error message.
File path – user specified file path location to save the tab delimited text file containing logged data	
Virtual channels in - array of clusters for all devices. Contains current device data to be recorded.	
Error in – standard error cluster comprised of error indicator, error code, and error message.	
Time stamp – time stamp for each loop iteration to include with each line of data acquired.	

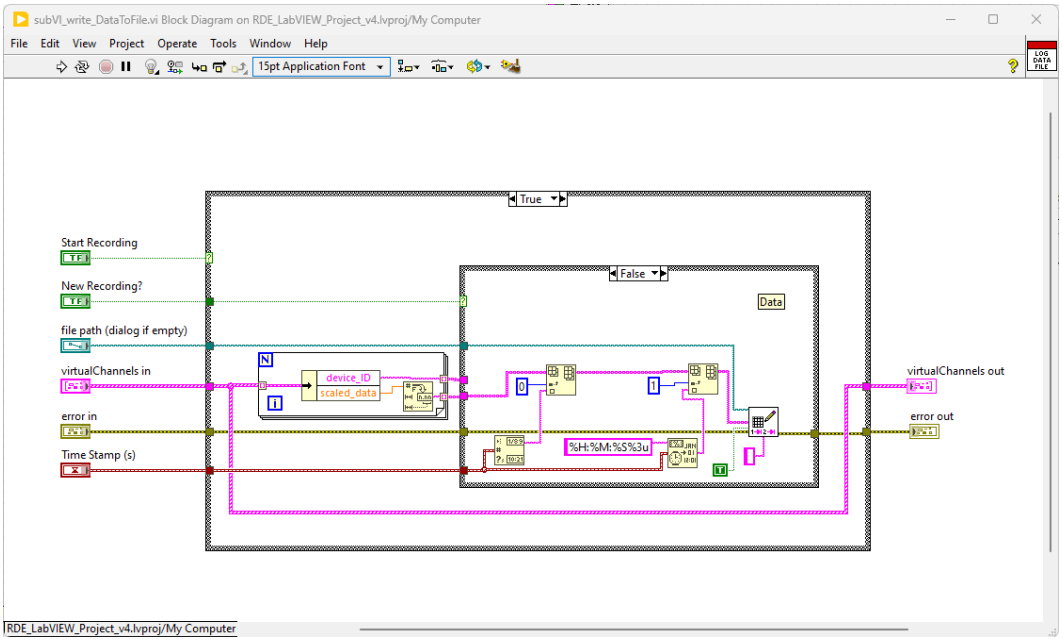


Figure 3.2.7: Write Data to File SubVI Block Diagram

3.2.8 subVI_read_testSequence_File.vi

This subroutine is designed to operate only when called by the user. It performs a single function of importing the test sequence initialization file as prepared in Excel which was converted into a readable format through a MATLAB script file. The data is imported as a 2D array of strings. It exists as 5 columns of data to mirror the test sequence template in Excel. It is noted that to utilize the content, the data must be transposed and strings associated with numeric data must be

converted into numeric arrays. Numeric contents of test sequence are converted from strings to numbers at the point of use in the automated sequence. Table 3.2.8 describes this SubVI's inputs and outputs, and Figure 3.2.8 shows its Block Diagram.

Table 3.2.8: SubVI_read_testSequence_File.vi Inputs and Outputs

Inputs	**Outputs**
File path – file path to the initialization file built by MATLAB that was generated from the test sequence file excel template.	Table control – 2D array of strings containing all information from test sequence file.
Error in – standard error cluster comprised of error indicator, error code, and error message.	Error out – standard error cluster comprised of error indicator, error code, and error message.

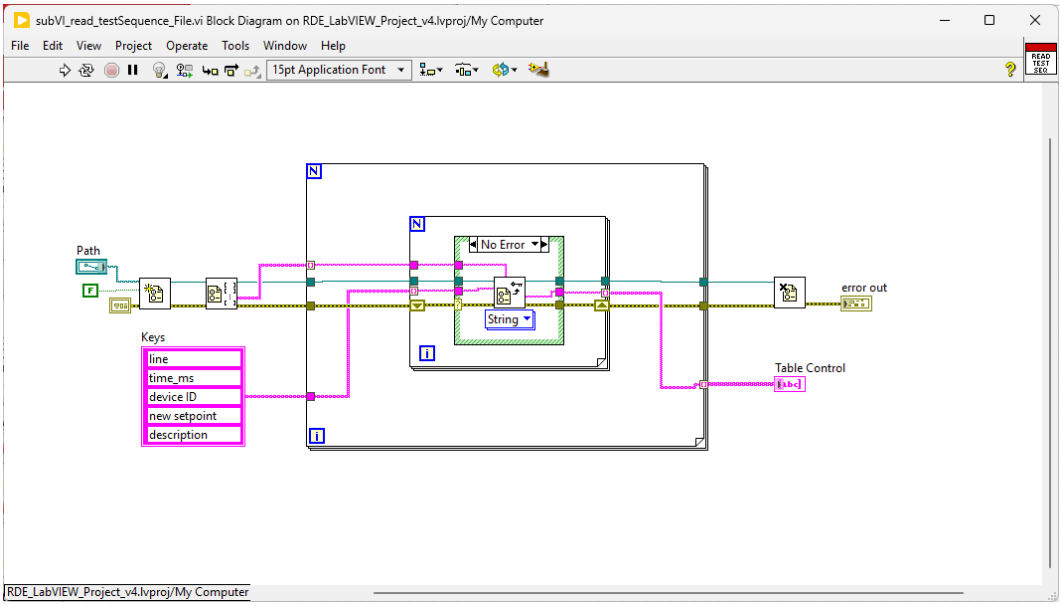


Figure 3.2.8: Read Test Sequence File SubVI Block Diagram

3.2.9 subVI_graphTestSequence.vi

This subroutine is designed to operate only when called by the user. It performs a single function of converting the 2D array of test sequence information into time series plot data for six specific valves considered critical during a test sequence. The time series arrays are created on a per millisecond time scale. The time scale of the plots is automatically scaled to 105% of the maximum time specified by the test sequence.

It is important to note that the data plotted is read from the tabulated data that is currently displayed in LabVIEW. Data in this table can be manually manipulated after a test sequence file is loaded. The intent of this design was to allow the user to make last minute adjustments to the test sequence and re-visualize the latest sequence that will run without having to prepare a new test sequence file. Table 3.2.9 describes this SubVI's inputs and outputs, and Figure 3.2.9 shows its Block Diagram.

Table 3.2.9: SubVI_graphTestSequence.vi Inputs and Outputs

Inputs	Outputs
Table control – 2D array of strings containing all information from test sequence file.	Chart cluster – cluster containing 5 arrays of time series valve position data corresponding to 5 critical system valves. Data for each array is valve position on a per millisecond basis.

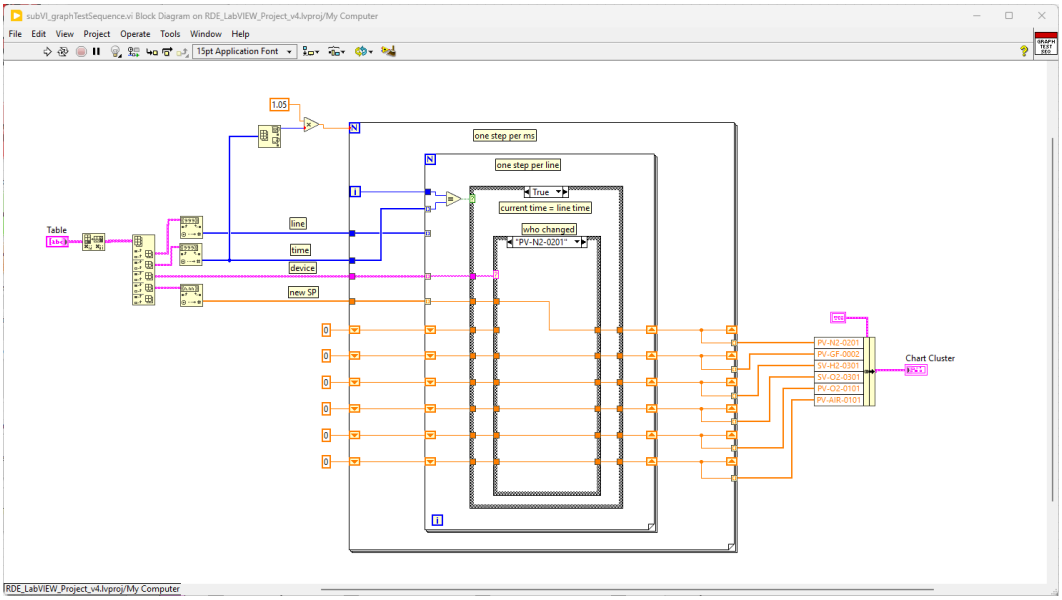


Figure 3.2.9: Graph Test Sequence SubVI Block Diagram

3.2.10 subVI_GraphPressureTrends.vi (and SubVI_GraphTemperatureTrends.vi)

This subroutine controls the graphing of trends of user specified device IDs. It is designed to input any device ID in any order at any time during operation. Each graph can accommodate up to 12 device IDs simultaneously. Each chart legend is updated in real time to match user specified device IDs. Chart Y-scales can be set to Autoscale or can be manually set to user specified range by directly adjusting the limits on the graph. The chart history length is default set to buffer 500 data points for each device ID. As the graphs are updated in sequence with the MainVI Front Panel frequency, the actual length of time buffered on each graph can be adjusted based on user preference. At a standard 20 Hz Front Panel refresh rate, this corresponds to approximately 25 seconds of buffered data on each graph. Each device ID being graphed also has the option to be temporarily hidden using the toggle button beside each device ID. Charting of each device continues in the background despite being hidden.

The Graph Pressure Trends and Graph Temperature Trends SubVIs are identical except for their title. Two instances of the same file exist so that the two SubVIs can operate simultaneously. These SubVIs are always active during standard operation although their Front Panel is only visible when requested by the user from the MainVI trend toggle buttons. Table 3.2.10 describes this SubVI's inputs and outputs, and Figure 3.2.10 shows its Block Diagram.

Table 3.2.10: SubVI_GraphPressureTrends.vi Inputs and Outputs

Inputs	Outputs
Open/Close FP – Boolean operator to open pop-up window from MainVI.	Waveform chart A – live chart display A trending data from device IDs as supplied.
Virtual channels in – array of clusters for all devices.	Waveform chart A plot.name – live chart legend display that updates with device IDs as supplied.
Time stamp – time stamp for each loop iteration to include with each line of data acquired.	Waveform chart B – live chart display B trending data from device IDs as supplied.
Clear history A – Boolean operator to clear chart contents from Chart A.	Waveform chart B plot.name – live chart legend display that updates with device IDs as supplied.
Clear history B - Boolean operator to clear chart contents from Chart B.	
Device IDs Chart A – array of Device IDs input from the pup-up window front panel to include on Chart A.	
Device IDs Chart B - array of Device IDs input from the pup-up window front panel to include on Chart B.	

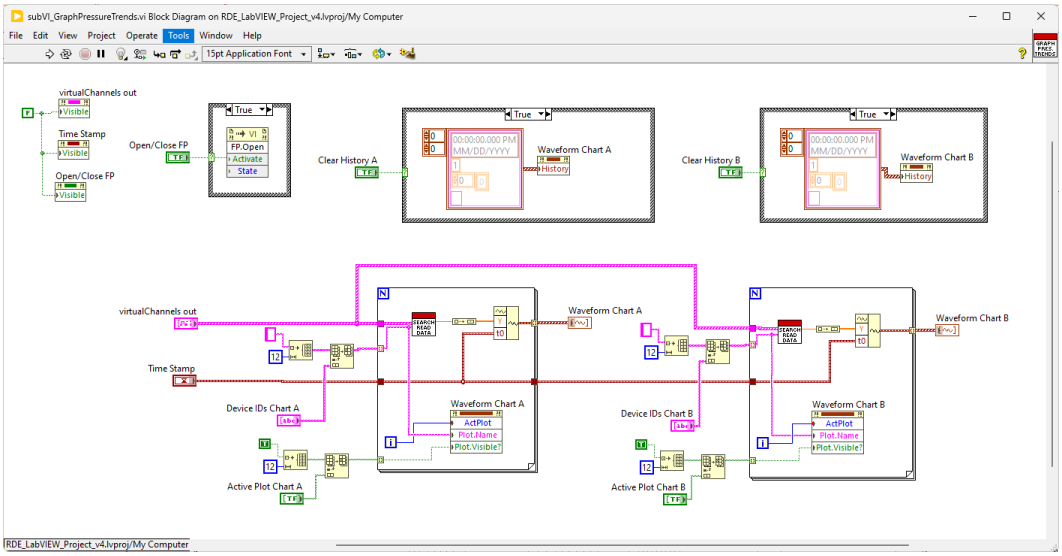


Figure 3.2.10: Graph Pressure Trends SubVI Block Diagram

3.2.11 subVI_clear_Tasks.vi

This subroutine is designed to only execute once at the end of a LabVIEW operating session. It performs an important task of closing each virtual channel task and clearing it from the DAQ memory. Without properly clearing this information from memory, the task can be saved between each LabVIEW session which will generate errors when a duplicate task of the same name and channel assignment attempts to be created. By design, this LabVIEW code is designed

to create new task assignments on every session based on the latest MIL file loaded into the system. Thus, these new tasks must be closed at the end of each LabVIEW session. Interlocks prevent the user from using the default LabVIEW abort button when connected to the DAQ. This requires the user to properly disconnect the DAQ when the LabVIEW session is complete. Table 3.2.11 describes this SubVI's inputs and outputs, and Figure 3.2.11 shows its Block Diagram.

Table 3.2.11: SubVI_clear_Tasks.vi Inputs and Outputs

Inputs	**Outputs**
Virtual channels in – array of clusters for all devices.	Error out – standard error cluster comprised of error indicator, error code, and error message.
Error in – standard error cluster comprised of error indicator, error code, and error message.	

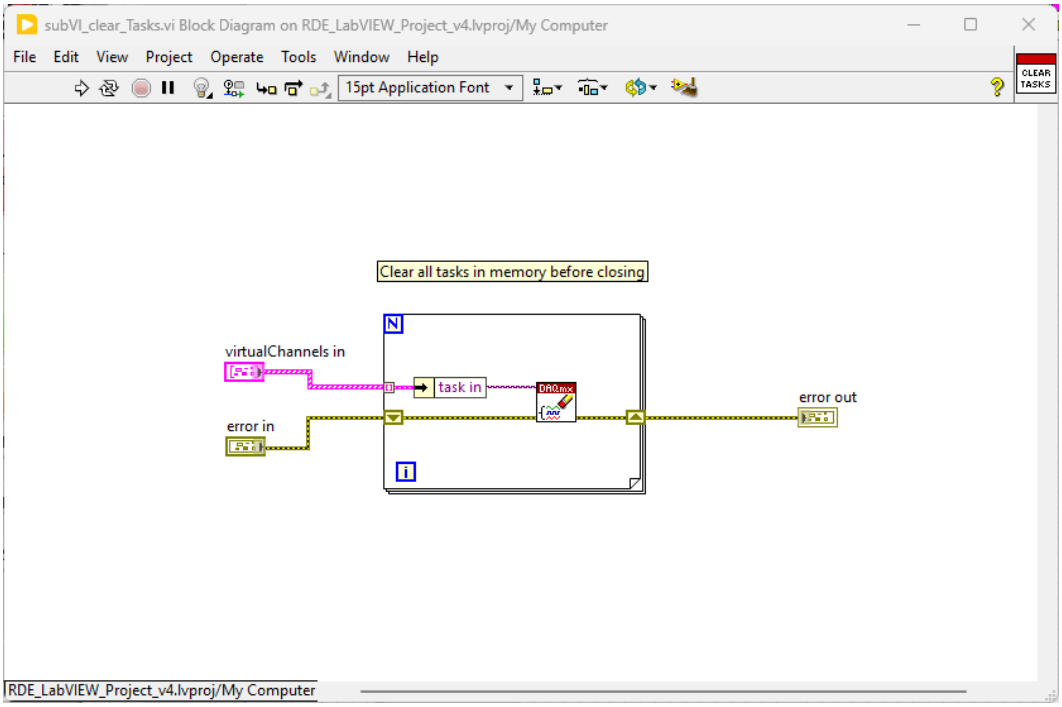


Figure 3.2.11: Clear Tasks SubVI Block Diagram

3.3 System Operating Procedure

A basic procedure for operating the MainVI system is provided below. This procedure assumes an Excel MIL configuration file has already been prepared and converted to an .ini initialization file through the MATLAB script file. It also assumes a similar Excel test sequence file has been prepared and converted to an .ini initialization file through its corresponding MATLAB script file. Operation of these MATLAB scripts is considered outside the scope of this procedure, but it is noted that this step does not require any special training beyond basic MATLAB experience.

1. Open the MainVI.
    a. The current program can be found via a shortcut on the BraunLab17 desktop: RDE_mainVI_v4.4.vi or can be loaded from the documents folder of the same computer. From the folders on the computer, version 4.4 is most up to date. When launching directly from the folders, launch "RDE_mainVI_v4.vi."
    b. When the VI is opened, it will automatically launch LabVIEW into its running state.

2. Load a configuration file and Connect the DAQ
    a. Choose an appropriate MIL configuration file from the drop down as shown in Figure 3.3.1. The file format must be a .ini initialization file.
    b. Press Connect PXIe button to establish connection with the DAQ. If connection is successful, indicators on the Front Panel should begin streaming data from their respective devices.

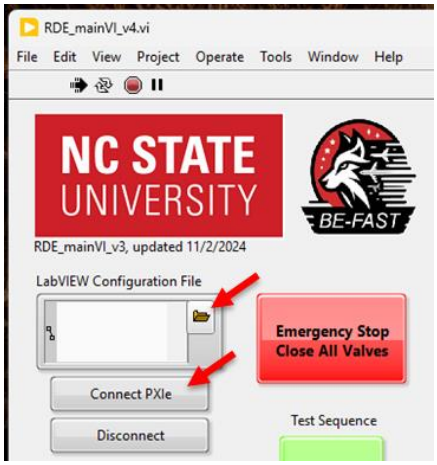


Figure 3.3.1: Loading the Configuration and Connecting the DAQ

3. Load a data logger output file location
    a. It is noted that a blank text file with desired file name may need to be generated in the desired target directory if a previous file does not exist. Once a file is loaded, the file name can be modified in the associated file path display box.
    b. The system can be operated without specifying a file path, but it recommended to do so before proceeding further.

4. Open Pressure Trend and Temperature Trend pop-up windows and define device IDs of interest.
    a. This step can be performed at any time.
    b. Device IDs can be modified at any time.
    c. To reset chart history, press "Clear History" button.
    d. At any time, right clicking on the plot can provide additional options such as rescaling X or Y axes and exporting current data buffer to Excel.
    e. Once trends are configured as desired, they can be closed or moved to a secondary screen for monitoring. Closing the pop-up window will not lose

configuration settings or trending data. Trends will continue to update in the background.

5. Import the test sequence
    a. Switch to the Test Sequence tab at the top of the screen as shown in Figure 3.3.2.
    b. Load the test sequence file in the appropriate dialog box. File format must be a .ini initialization file.
    c. Press the “Import Test Sequence” button to load the contents of the file. If successful, the test sequence table should populate with the desired information.
    d. Press the “Update Visualization” button to visualize the critical valve positions in time.
    e. If needed, make manual modifications to the test sequence table as displayed and repeat updating the visualization.

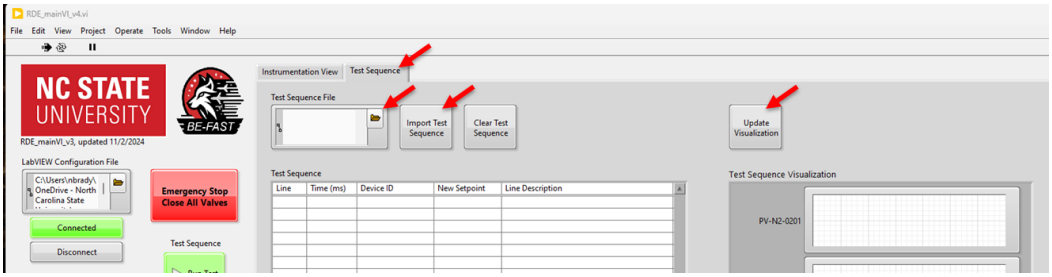


Figure 3.3.2: Importing the Test Sequence

6. Prepare manual valve and regulator adjustments prior to automated sequence
    a. Switch back to the Instrumentation View tab.
    b. Toggle valves open or closed as desired to perform manual purge or checkout activities. Valves are manually controlled by pressing the toggle button above the associated valve as shown in Figure 3.3.3.
    c. Set desired regulator pressures for each regulator device using the input dialog box as shown in Figure 3.3.3.
    d. Set venturi throat diameters and densities for each gas to enable live computation of mass flow rate for each gas.

    IMPORTANT NOTE: If at any point, unexpected system activity occurs or a mistake is made while entering regulator pressure or a valve is opened by mistake, the EMERGENCY STOP, CLOSE ALL VALVES button can be used to immediately close all regulators and valves. However, if for any reason it is believed that communication with the DAQ may be compromised, the user is instructed to use a physical emergency stop device to shut down the system. LabVIEW can only control what it has active communication with. A physical emergency stop device will guarantee valve closure by removing control circuit power allowing systems to default to a safe state.

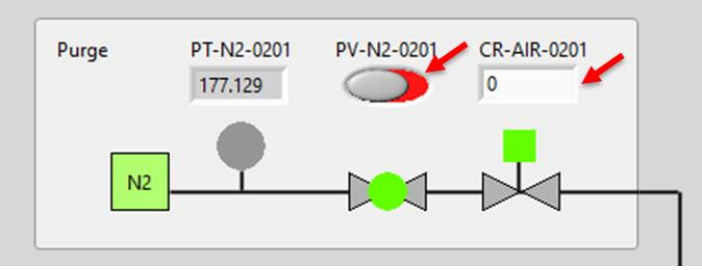


Figure 3.3.3: Manually Actuating Valves and Setting Regulator Pressures

7. Run the test sequence.
    a. Once all system checks have been performed and found satisfactory, press the "Run Test" button to initiate the automated test sequence. The test sequence will execute according to the list actually shown in the test sequence tab. The test sequence clock will start as shown by Figure 3.3.4.
    b. Test execution can be monitored from either system tab.
        i. On the Instrumentation View tab, valves and regulators will reflect their actual status as they open and close according to the specified test sequence. Pressure indicators will turn green as they cross a specified threshold.
        ii. On the Test Sequence tab, the next line to be executed will be highlighted on the test sequence table. This provides live feedback of the system status during an automated sequence.
    c. Upon completion of the test sequence, the system will return to manual mode.

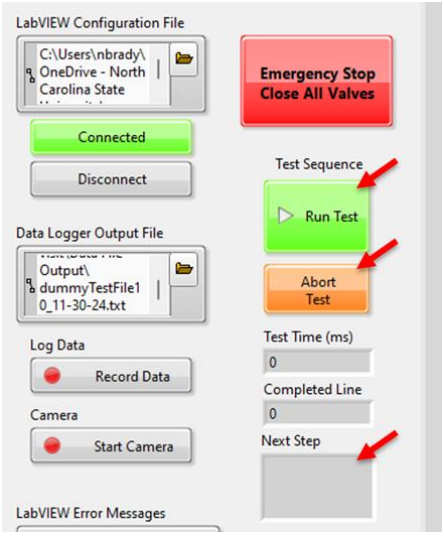


Figure 3.3.4: Initiating the Test Sequence

IMPORTANT NOTE: At any point during the test sequence, the EMERGENCY STOP or ABORT buttons can be utilized to immediately stop the test sequence and close all valves and regulators. As stated above, if it is believed that DAQ communication may be compromised, the user is instructed to use a physical emergency stop device.

8. To run another test, repeat steps 3 – 7.
    a. The DAQ can remain connected, and the system can operate in manual mode between test sequences.
    b. A new test sequence file can be loaded, or the current test sequence can be modified directly in the Test Sequence tab.

9. If testing is complete, shut down LabVIEW.
    a. Disconnect the DAQ using the "Disconnect" button shown in Figure 3.3.5.
    b. Once the system disconnects, it will switch into Editor Mode. It is now safe to close the MainVI window and close LabVIEW.

    IMPORTANT NOTE: Premature closure of the LabVIEW window without properly disconnecting the DAQ may cause memory leakage errors upon restart. Disconnecting the DAQ performs a critical clearing of channel memory to prepare LabVIEW for its next session.

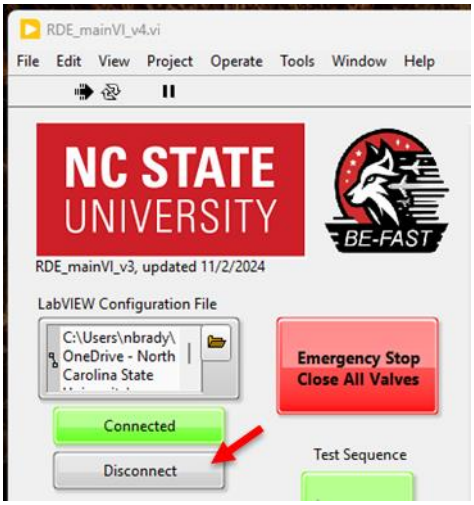


Figure 3.3.5: Disconnecting the DAQ

3.4 System Checkout Test Results

With the MainVI and auxiliary SubVIs completed, several tests were performed to verify several critical functions. These included data output file verification, loop frequency evaluation, digital output capability to physical devices, and analog input capability from a physical sensor.

Data logging capability was evaluated using the simulated DAQ devices configured in NI MAX. The MIL was prepared using a feasible set of channel inputs along with representative device calibration and scaling information. An exhaustive representative test sequence was prepared to actuate all outputs available. The test sequence included setting of all regulator settings, actuation of pre-detonation activities, ignition, purge cycling before and after the test, and automated data logger and camera activation. The test was executed according to the procedure outlined in section 3.3 of this report. The data logger file was successfully created as planned.

Table 3.4.1 provides a truncated view of the first 10 lines of data acquired for the first 8 devices recorded. In total, all 51 devices were recorded as defined by the MIL. In 30.084 seconds, a total of 2228 lines of data were acquired, averaging 0.014±0.002 seconds per data set. This was deemed an acceptable data acquisition rate as it is on the same order of magnitude or faster than most of the system's actuation devices.

Table 3.4.1: Data Logger Capability Test Result Subset

date	time	PT-N2-0201	PT-GF-0001	PT-GF-0002	PT-GF-0003	PT-GF-0004	PT-H2-0301	PT-O2-0301	PT-O2-0101	…
12/1/2024	33:54.991	1819.697	922.177	942.861	879.949	923.088	936.660	896.898	935.606	…
12/1/2024	33:55.002	1806.085	917.676	943.654	878.220	919.633	935.560	896.777	922.096	…
12/1/2024	33:55.012	1823.359	914.563	938.259	879.392	921.750	934.240	894.813	918.703	…
12/1/2024	33:55.023	1817.621	912.776	932.800	870.954	919.570	941.658	898.288	929.287	…
12/1/2024	33:55.033	1809.076	907.720	934.291	866.442	920.598	928.205	889.345	914.282	…
12/1/2024	33:55.043	1813.715	907.227	931.816	866.354	921.874	936.315	888.891	923.621	…
12/1/2024	33:55.056	1811.274	905.378	926.325	871.013	915.306	926.067	881.942	915.434	…
12/1/2024	33:55.067	1804.926	907.813	934.323	863.014	923.026	930.845	886.082	918.018	…
12/1/2024	33:55.077	1788.934	902.604	926.293	861.813	915.586	931.851	891.097	916.835	…
12/1/2024	33:55.087	1803.461	897.579	934.006	861.110	908.489	926.633	882.607	906.531	…
…	…	…	…	…	…	…	…	…	…	…

Figures 3.4.1 and 3.4.2 were prepared from this dataset to show the pressure regulator setpoints and critical valve positions respectively. These charts demonstrate successful assignment of each output device setpoint throughout the test sequence at time scales appropriate to their function.

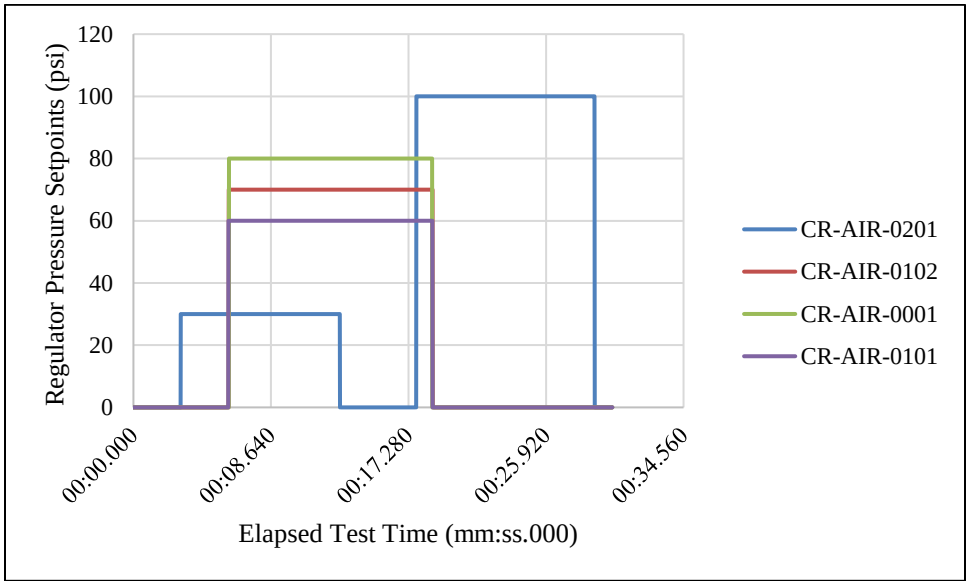


Figure 3.4.1: Data Logger Capability Test Pressure Regulator Data

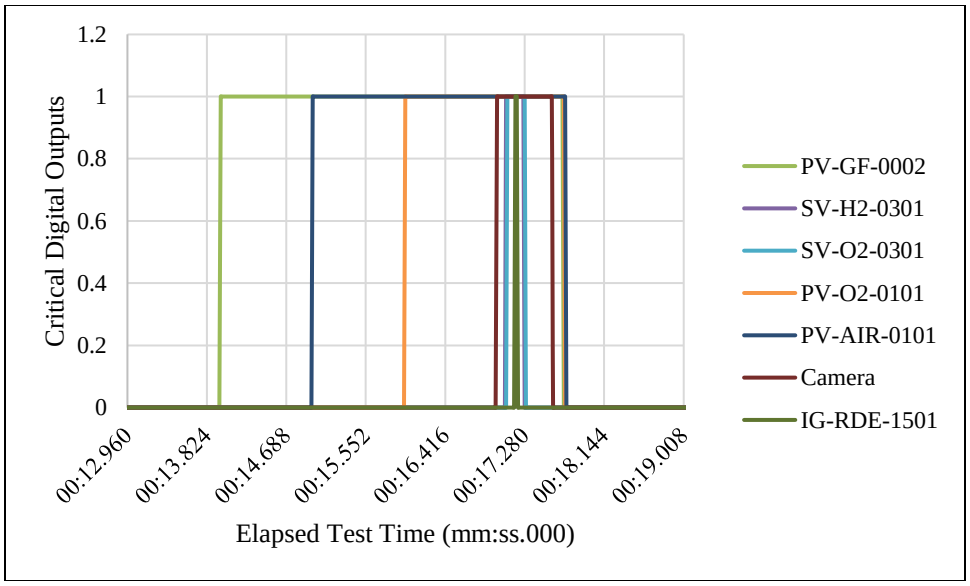


Figure 3.4.2: Data Logger Capability Test Valve Position Data

It was attempted to repeat this test with the physical DAQ device to get validation that the simulated DAQ behavior matched real device behavior. However, it was found that the physical DAQ operated at a significantly slower primary loop iteration rate than what was previously experienced with the simulated DAQ devices. Further inquiry discovered that the physical DAQ communication rate was severely limited due to the choice of individual task assignment architecture. The calling of each task individually from within the primary loop caused the channel data acquisition to be started and stopped for each task. The starting and stopping of each channel created significant delays in loop frequency.

As briefly discussed in the SubVI Architecture sections 3.2.2 and 3.2.6, it was found that starting and stopping of virtual channels should occur outside the primary loop to prevent communication delay. However, multiple channels within a single DAQ module cannot be individually active. Rather, the entire DAQ module (card) must be started and stopped. This requires channels and subsequent tasks to be grouped by DAQ module such that each group can be started and stopped outside the primary loop. This modification to DAQmx architecture will be a focus of future study to improve communication speed with the physical DAQ devices.

Despite communication delays with the physical DAQ device, communication was sufficient to perform MainVI system verifications with physical devices. Digital output connectivity was assessed based on the MainVI's capability to actuate the status LEDs on each digital output's associated control relay. Testing was able to successfully verify that the MainVI correctly activated each digital output's control relay to the ON or OFF position as commanded. Figure 3.4.3 demonstrates a subset of these test results.

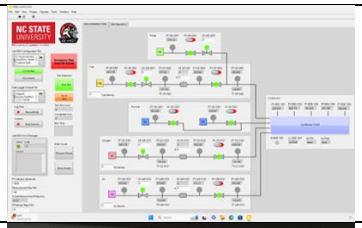
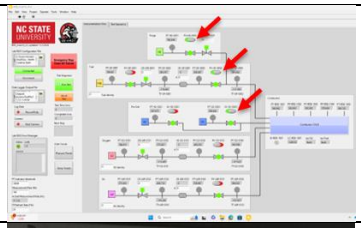
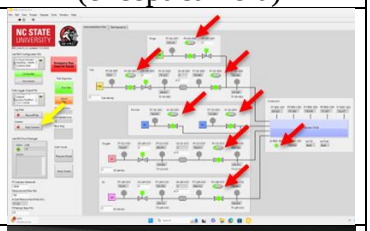
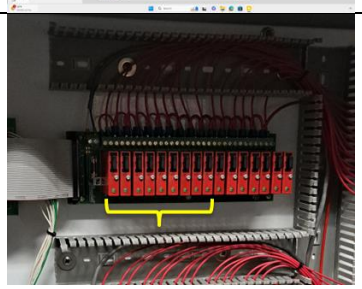
	Test 1 All Digital Outputs OFF	Test 2 3 Digital Outputs ON	Test 3 All Digital Outputs ON (except camera)
MainVI Front Panel			
Digital Output Control Relays			
Test Result	All control relays remained OFF as commanded.	3 selected control relays switched ON as commanded.	All 8 selected control relays switched ON as commanded.

Figure 3.4.3: Digital Output Control Test Results

To confirm analog input capability, a single pressure transducer was temporarily connected to a test apparatus as shown in Figure 3.4.4. Compressed air was supplied to a pressure regulator which provided static pressure to the pressure transducer. A manual bleed off valve was used to vary the static pressure applied to the pressure transducer. The analog input was successfully read into the MainVI and plotted on the pressure trend chart. The data for the temporary test was captured by exporting the pressure trend chart data directly to Excel. Figure 3.4.5 shows a subset of the results of this test. The manual bleed off valve was closed three times in sequence resulting in three corresponding rises in static pressure on the device.

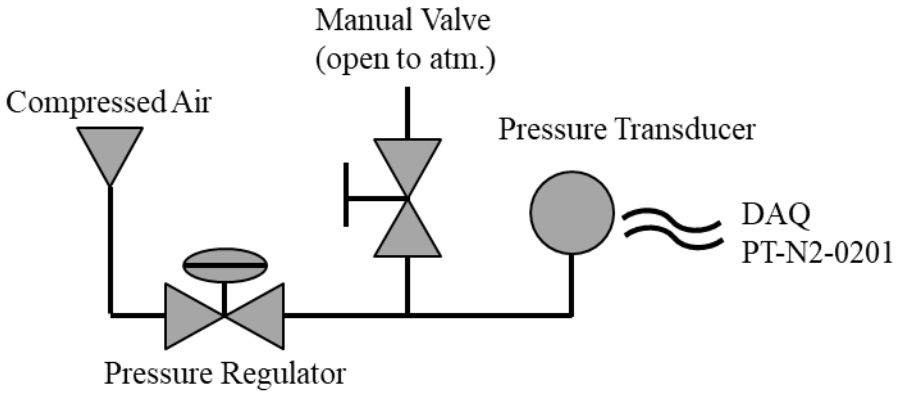


Figure 3.4.4: Analog Input Capability Test Apparatus

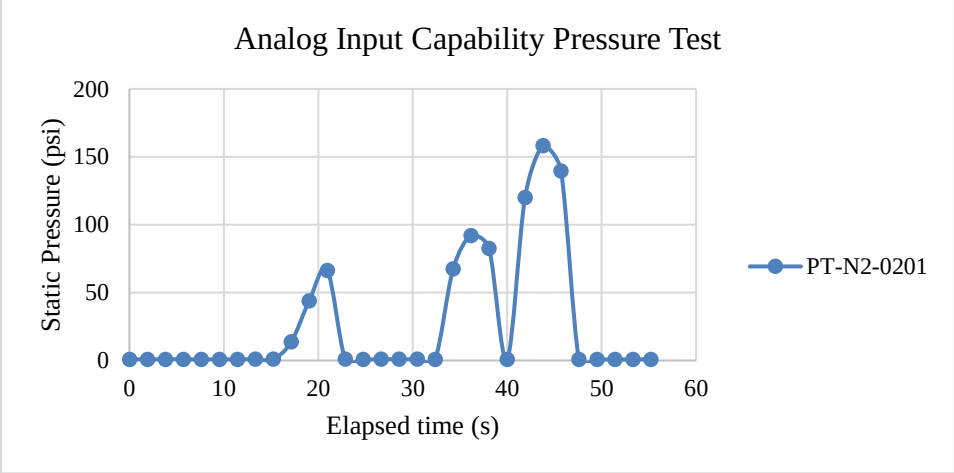


Figure 3.4.5: Analog Input Capability Pressure Test Results

Discussion

An RDE system is a complex device to control in LabVIEW. Its complexity stems from its assortment of many data types (analog inputs, analog outputs, digital outputs) as well as the number of channels to be viewed and controlled simultaneously. In addition, there is an underlying need to maintain an efficient programming scheme so that automated system control can progress quickly enough to keep up with the physical detonation process.

It was the intent of this project to design, develop, and test a LabVIEW control system to automate an RDE test stand. This project delivered a VI control architecture with supporting SubVI library capable of comprehensively controlling all devices necessary for an RDE hot fire experiment. Sufficient code flexibility was incorporated within the design to ensure the final LabVIEW system can grow with the RDE development effort. The overall system was developed through extensive testing to debug and properly interlock devices for a seamless user experience. The system was designed to require minimal training for new users for basic operation. To verify the design, physical testing was performed with the DAQ on site to ensure communication could be properly established and the system would behave as intended. This testing successfully demonstrated multiple critical aspects of the system performed as anticipated.

Future development of this LabVIEW test system should include reassessment of channel assignments into a DAQ module grouped scheme as opposed to the current individual channel assignment scheme to mitigate primary loop iteration delays. The necessity of this improvement is described in more detail in sections 3.2 and 3.4. Test data acquired during the simulated DAQ device verification runs demonstrates the overall code architecture is capable of attaining sufficient primary loop iteration speed. However, delays resulting from connection to the physical DAQ necessitate the modification of channel assignment architecture. Some initial activities towards this code development have begun but have not reached full implementation. This improvement will be an important step towards an engine hot fire demonstration.

A less critical future improvement to the LabVIEW system should explore offline data acquisition through a dedicated data logging device. While data logging capability was successfully implemented into the VI architecture presented here, it was found that Windows clock speeds are characteristically inconsistent at millisecond time scales. As presented in section 3.2, time stamps between loop iterations had a standard deviation of ±2ms. While this may appear inconsequential, it is a fairly large sample rate inconsistency in comparison to the time between iterations. The root of this inconsistency stems from the data being logged within a Windows operating system. To remove this irregularity, an offline data acquisition module can be explored to improve accuracy between sampling times. A dedicated data acquisition module can be expected to perform with better sample rate consistency. It will also need to be evaluated if this improved time scale consistency is required during post test data analysis.

The LabVIEW system presented in this work is a critical component to enable success of the RDE test stand. It provides a foundational control architecture to perform required automation of RDE devices for experimentation. The work presented here will have a long-lasting impact as a fundamental element of this continued research effort.

References

[1] J. Braun, G. Paniagua, Rotating detonation combustor operability and aero-thermal performance with an integrated diverging nozzle, Applied Thermal Engineering 249 (2024) 123126, https://doi.org/10.1016/j.applthermaleng.2024.123126.

[2] J. Braun, B.H. Saracoglu, G. Paniagua, Unsteady performance of rotating detonation engines with different exhaust nozzles, J. Propul. Power 33 (1) (Jan. 2017) 121–130, https://doi.org/10.2514/1.B36164.

[3] I. H. Koo, K.H. Lee, M. S. Kim, H. S. Han, H Kim, J. Y. Choi, Effects of Injector Configuration on the Detonation Characteristics and Propulsion Performance of Rotating Detonation Engine (RDE), Aerospace 2023, 10, 949, https://doi.org/10.3390/aerospace10110949.

Appendix A

Section	Long Description	P&ID name	signal_type	terminal_config	device_identifier	channel	signal_unit	theo_min	theo_max	cal_zero	cal_slope	eng_scale
purge	N2 upstream pneumatic valve pressure	PT-N2-0201	AI	Differential	PXIe-6375	ai16	volts	0	10	0.00	1.000	200
purge	N2 pneumatic shutoff valve	PV-N2-0201	DO	none	PXIe-6535	port0/line0	one channel for each line	0	1	0.00	1.000	1
purge	computer regulator driving PR-N2-0201	CR-AIR-0201	AO	RSE	PXIe-6739	ao0	volts	0	24	0.00	1.000	100
fuel	fuel upstream pneumatic valve pressure	PT-GF-0001	AI	Differential	PXIe-6375	ai17	volts	0	10	0.00	1.000	200
fuel	fuel pneumatic shutoff valve	PV-GF-0001	DO	none	PXIe-6535	port0/line1	one channel for each line	0	1	0.00	1.000	1
fuel	computer regulator driving PR-GF-0001	CR-AIR-0001	AO	RSE	PXIe-6739	ao4	volts	0	24	0.00	1.000	100
fuel	fuel upstream venturi pressure	PT-GF-0002	AI	Differential	PXIe-6375	ai18	volts	0	10	0.00	1.000	200
fuel	fuel upstream venturi temperature	TT-GF-0001	TC	none	PXIe-4302	ai0	Deg C	0	0.055	0.00	1.000	17.88
fuel	fuel downstream venturi pressure	PT-GF-0003	AI	Differential	PXIe-6375	ai19	volts	0	10	0.00	1.000	200
fuel	fuel pneumatic valve upstream purge tie in	PV-GF-0002	DO	none	PXIe-6535	port0/line2	one channel for each line	0	1	0.00	1.000	1
fuel	fuel pressure at engine inlet	PT-GF-0004	AI	Differential	PXIe-6375	ai20	volts	0	10	0.00	1.000	200
fuel	fuel temperature engine inlet	TT-GF-0002	TC	none	PXIe-4302	ai4	Deg C	0	0.055	0.00	1.000	17.88
pre-det	pre-det H2 upstream pneumatic valve pressure	PT-H2-0301	AI	Differential	PXIe-6375	ai21	volts	0	10	0.00	1.000	200
pre-det	pre-det H2 shutoff valve	SV-H2-0301	DO	none	PXIe-6535	port0/line3	one channel for each line	0	1	0.00	1.000	1
pre-det	pre-det O2 upstream pneumatic valve pressure	PT-O2-0301	AI	Differential	PXIe-6375	ai22	volts	0	10	0.00	1.000	200
pre-det	pre-det O2 shutoff valve	SV-O2-0301	DO	none	PXIe-6535	port0/line4	one channel for each line	0	1	0.00	1.000	1
oxygen	oxygen upstream regulator pressure	PT-O2-0101	AI	Differential	PXIe-6375	ai23	volts	0	10	0.00	1.000	200
oxygen	computer regulator driving PR-O2-0101	CR-AIR-0101	AO	RSE	PXIe-6739	ao8	volts	0	24	0.00	1.000	100
oxygen	oxygen upstream venturi pressure	PT-O2-0102	AI	Differential	PXIe-6375	ai32	volts	0	10	0.00	1.000	200
oxygen	oxygen upstream venturi temperature	TT-O2-0101	TC	none	PXIe-4302	ai8	Deg C	0	0.055	0.00	1.000	17.88
oxygen	oxygen downstream venturi pressure	PT-O2-0103	AI	Differential	PXIe-6375	ai33	volts	0	10	0.00	1.000	200
oxygen	oxygen pneumatic shutoff valve	PV-O2-0101	DO	none	PXIe-6535	port0/line5	one channel for each line	0	1	0.00	1.000	1
oxygen	oxygen temperature engine inlet	TT-O2-0103	TC	none	PXIe-4302	ai12	Deg C	0	0.055	0.00	1.000	17.88
oxygen	oxygen pressure engine inlet	PT-O2-0104	AI	Differential	PXIe-6375	ai34	volts	0	10	0.00	1.000	200
air	air upstream regulator pressure	PT-AIR-0101	AI	Differential	PXIe-6375	ai35	volts	0	10	0.00	1.000	200
air	computer regulator driving PR-AIR-0102 driving PR-AIR-0101	CR-AIR-0102	AO	RSE	PXIe-6739	ao16	volts	0	24	0.00	1.000	100
air	air upstream venturi pressure	PT-AIR-0102	AI	Differential	PXIe-6375	ai36	volts	0	10	0.00	1.000	200
air	air upstream venturi temperature	TT-AIR-0101	TC	none	PXIe-4302	ai16	Deg C	0	0.055	0.00	1.000	17.88
air	air downstream venturi pressure	PT-AIR-0103	AI	Differential	PXIe-6375	ai37	volts	0	10	0.00	1.000	200
air	air pneumatic shutoff valve	PV-AIR-0101	DO	none	PXIe-6535	port0/line6	one channel for each line	0	1	0.00	1.000	1
air	air temperature engine inlet	TT-AIR-0103	TC	none	PXIe-4302	ai20	Deg C	0	0.055	0.00	1.000	17.88
air	air pressure engine inlet	PT-AIR-0104	AI	Differential	PXIe-6375	ai38	volts	0	10	0.00	1.000	200
engine	engine pressure location 0	PT-RDE-1501	AI	Differential	PXIe-6375	ai39	volts	0	10	0.00	1.000	200
engine	engine pressure location 1	PT-RDE-1502	AI	Differential	PXIe-6375	ai48	volts	0	10	0.00	1.000	200
engine	engine pressure location 2	PT-RDE-1503	AI	Differential	PXIe-6375	ai49	volts	0	10	0.00	1.000	200
engine	engine pressure location 3	PT-RDE-1504	AI	Differential	PXIe-6375	ai50	volts	0	10	0.00	1.000	200
engine	engine pressure location 4	PT-RDE-1505	AI	Differential	PXIe-6375	ai51	volts	0	10	0.00	1.000	200
camera	camera start/stop trigger	Camera	DO	none	PXIe-6535	port0/line7	one channel for each line	0	1	0.00	1.000	1
load cell	load cell AI	LC-RDE-1501	AI	Differential	PXIe-6375	ai52	volts	0	24	0.00	1.000	1
ignitor	gas line ignitor	IG-RDE-1501	DO	none	PXIe-6535	port1/line0	one channel for each line	0	1	0.00	1.000	1
purge	purge regulator feedback indicator	PR-N2-0201i	AI	Differential	PXIe-6375	ai52	volts	0	24	0.00	1.000	1
fuel	fuel regulator feedback indicator	PR-GF-0001i	AI	Differential	PXIe-6375	ai53	volts	0	24	0.00	1.000	1
oxygen	oxygen regulator feedback indicator	PR-O2-0101i	AI	Differential	PXIe-6375	ai54	volts	0	24	0.00	1.000	1
air	air regulator feedback indicator	PR-AIR-0101i	AI	Differential	PXIe-6375	ai55	volts	0	24	0.00	1.000	1
purge	purge valve feedback indicator	PV-N2-0201i	AI	Differential	PXIe-6375	ai64	volts	0	24	0.00	1.000	1
fuel	fuel upstream valve feedback indicator	PV-GF-0001i	AI	Differential	PXIe-6375	ai65	volts	0	24	0.00	1.000	1
fuel	fuel downstream valve feedback indicator	PV-GF-0002i	AI	Differential	PXIe-6375	ai66	volts	0	24	0.00	1.000	1
pre-det	pre-det H2 valve feedback indicator	SV-H2-0301i	AI	Differential	PXIe-6375	ai67	volts	0	24	0.00	1.000	1
pre-det	pre-det O2 valve feedback indicator	SV-O2-0301i	AI	Differential	PXIe-6375	ai68	volts	0	24	0.00	1.000	1
oxygen	oxygen valve feedback indicator	PV-O2-0101i	AI	Differential	PXIe-6375	ai69	volts	0	24	0.00	1.000	1
air	air valve feedback indicator	PV-AIR-0101i	AI	Differential	PXIe-6375	ai70	volts	0	24	0.00	1.000	1

Figure A1: Master Instrumentation List (MIL) Excel Example

Test Sequence				
Test	**template**			
Description	**Description of the test sequence below**			
date	**11/25/2024**			
line	**time (ms)**	**device**	**new value**	**line description**
1	0	CR-AIR-0201	0	close N2 regulator
2	0	CR-AIR-0001	0	close fuel regulator
3	0	CR-AIR-0101	0	close O2 regulator
4	0	CR-AIR-0102	0	close Air regulator
5	0	PV-GF-0002	0	close fuel downstream valve
6	0	SV-H2-0301	0	close pre-det H2 downstream valve
7	0	SV-O2-0301	0	close pre-det O2 downstream valve
8	0	PV-O2-0101	0	close O2 downstream valve
9	0	PV-AIR-0101	0	close air downstream valve
10	0	PV-N2-0201	0	close N2 upstream valve
11	0	PV-GF-0001	0	close fuel upstream valve
12	3000	Log Data	1	start data recording
13	3010	PV-N2-0201	1	open N2 upstream valve
14	3010	PV-GF-0001	1	open fuel upstream valve
15	6000	CR-AIR-0201	30	open N2 purge
16	9000	CR-AIR-0101	100	set O2 pressure
17	9000	CR-AIR-0102	100	set air pressure
18	9000	CR-AIR-0001	100	set fuel pressure
19	16000	CR-AIR-0201	0	turn off purge
20	17000	PV-GF-0002	1	open fuel flow
21	18000	PV-AIR-0101	1	open air flow
22	19000	PV-O2-0101	1	open O2 flow
23	20000	Camera	1	start camera recording
24	20100	SV-H2-0301	1	open pre-det H2
25	20100	SV-O2-0301	1	open pre-det O2
26	20200	IG-RDE-1501	1	spark ignitor
27	20230	IG-RDE-1501	0	stop ignitor
28	20300	SV-H2-0301	0	close pre-det H2
29	20300	SV-O2-0301	0	close pre-det O2
30	20600	Camera	0	stop camera recording
31	20730	PV-GF-0002	0	close fuel flow
32	20730	PV-O2-0101	0	close O2 flow
33	20730	PV-AIR-0101	0	close air flow
34	20800	CR-AIR-0201	100	open N2 purge
35	21800	CR-AIR-0001	0	close fuel regulator
36	21800	CR-AIR-0101	0	close O2 regulator
37	21800	CR-AIR-0102	0	close Air regulator
38	32000	CR-AIR-0201	0	close N2 purge
39	33000	PV-GF-0001	0	close fuel upstream valve
40	33000	PV-N2-0201	0	close N2 upstream valve
41	33100	Log Data	0	end data recording

Figure A2: Test Sequence Excel Template Example